

第4章 信息系统设计

- 4.1 信息系统设计概述
- 4.2 数据库设计和编码设计
- 4.3 人机界面设计
- 4.4 功能设计的基本方法
- 4.5 基于UML的功能设计方法
- 思考题



4.1 信息系统设计概述

■ 4.1.1 系统设计的任务

■ 确定系统功能结构

■ 功能模块的程序组成。

- 依据系统分析文档资料实现；

■ 功能模块间的联结方式。

- 以构成一个最好的系统机内结构为目标。

■ 表达设计成果

- 使用一定的工具表达所设计的成果。

■ 详细设计与实现

■ 数据库的详细设计

■ 编码设计

■ 输入/输出界面（人—机界面）设计等



4.1.2 系统设计步骤

- 第一步：数据库的物理设计。
- 第二步：编码设计。
- 第三步：界面设计。
- 第四步：系统功能设计。
- 根据总体方案及系统分析的实际需求：
 - 考虑数据库的**存储方式**（集中式或分布式）；
 - 考虑物理库的**数量**及每一个物理库所包含的数据库表的内容；
 - 从查询处理效率等方面考虑数据库表的**索引**问题等等。



4.1.2 系统设计步骤

- 第一步：数据库的物理设计。
 - 第二步：编码设计。
 - 第三步：界面设计。
 - 第四步：系统功能设计。
-
- 编码的重要性：
 - 与数据的规范性和交换性密切相关；
 - 编码的永久性：
 - 被设计后，其处理将会永久使用并常驻于数据库中
 - 给出具有永久性、稳定性的编码设计方案。



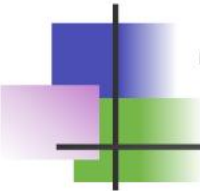
4.1.2 系统设计步骤

- 第一步：数据库的物理设计。
- 第二步：编码设计。
- 第三步：界面设计。
- 第四步：系统功能设计。
 - 构造良好的界面；
 - 程序设计之前完成。



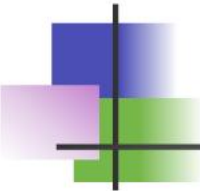
4.1.2 系统设计步骤

- 第一步：数据库的物理设计。
- 第二步：编码设计。
- 第三步：界面设计。
- **第四步：系统功能设计。**
 - 采用“自顶向下”的原则进行模块的划分与设计；
 - 使模块内部以及模块间的连接合理。
 - 设计结果的表达方式有两种：
 - **结构图**：描述模块的功能和模块之间的信息交换；
 - **Rational Rose工具**。
 - 用一组评价标准对设计结果进行评价；
 - 用一组设计原则对系统模块进行优化设计。



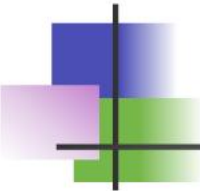
4.1.3 系统设计评价标准

- 1、运行效率
- 2、可靠性
- 3、可修改性
- 4、灵活性
- 5、通用性
- 6、实用性
- 前3项是最重要的评价标准



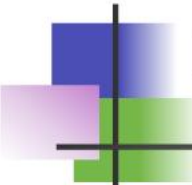
1、运行效率

- 处理能力
- 运行时间
- 响应时间
 - 信息系统中运行时间为：
 - 数据的操纵、分类、检索



1、运行效率

- 响应系统处理的因素：
 - ①系统中间文件的数量
 - ②数据文件的检索次数
 - ③外存访问次数
 - ④程序的调用和其它的系统开销所花费的时间
 - ⑤程序的执行时间



2、可靠性

- 分类：软件、硬件。
- 增强考虑可靠性的几个措施：
 - ①校验码
 - 例：银行账号 10位，最后1位是校验码
 - ②批量控制
 - 预定批处理事务的总数
 - 若其与人工计算总数不一致则有错，计算机不予处理。
 - ③建立运行日志和信息追踪
 - ④文件的存取限制
 - ⑤可靠性指标

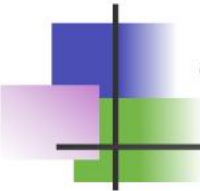


■可靠性指标:

- 平均故障间隔时间 (平均发生2次故障的间隔时间)
 - (Mean_Time_Between_Failures, —MTBF)

$$MTBF = \sum_{i=1}^n (t_{i+1} - t_i) / (n - 1)$$

- 平均修复时间
 - (Mean_Time_To_Repairs, —MTTR)
 - 平均每次所用的修理时间 (排除故障时间)
- 系统的有效性=MTBF/ (MTBF+MTTR)
 - 介于0~1, 越接近1, 系统的有效性越高



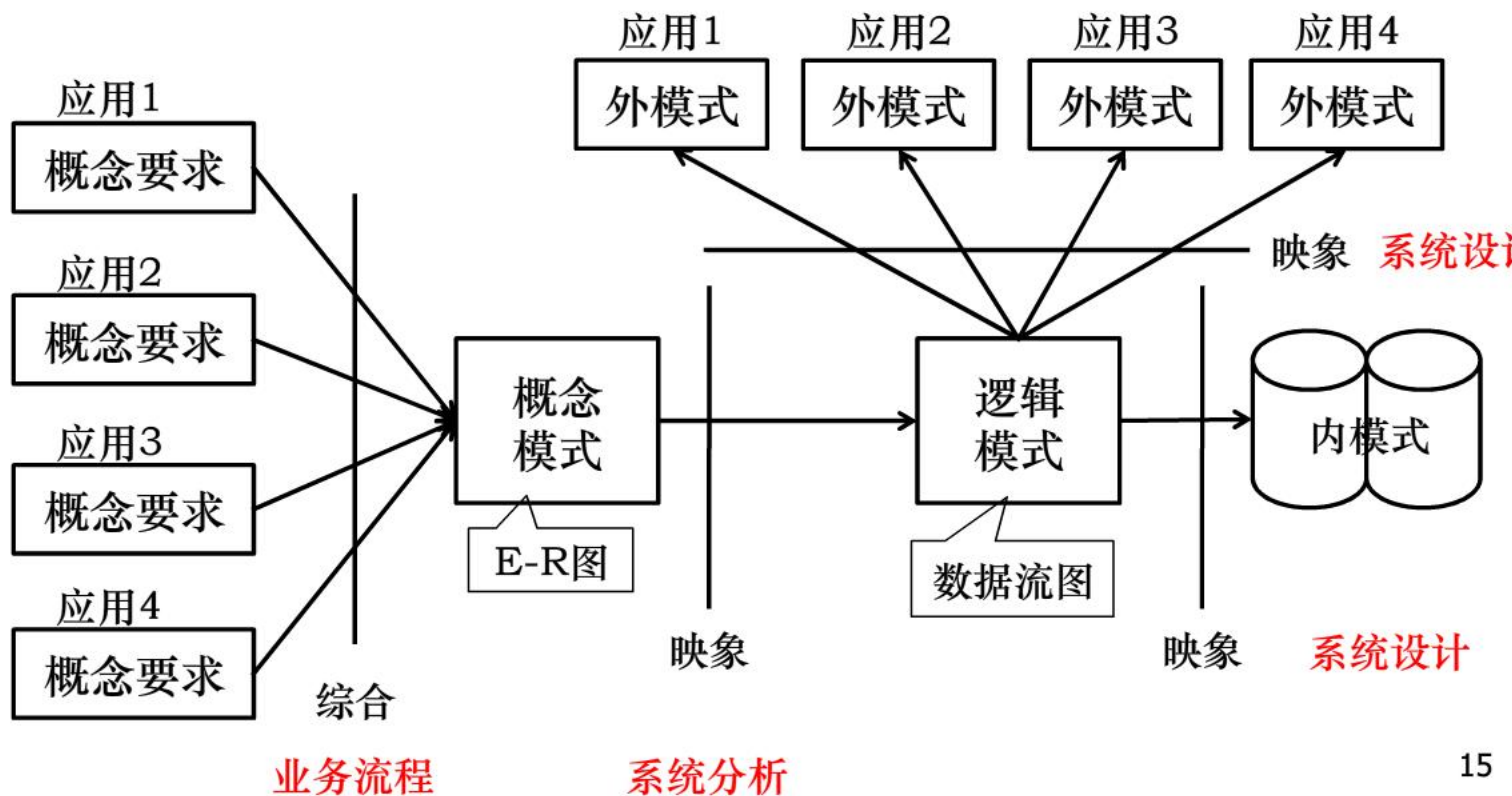
3、可修改性

- 必要性：

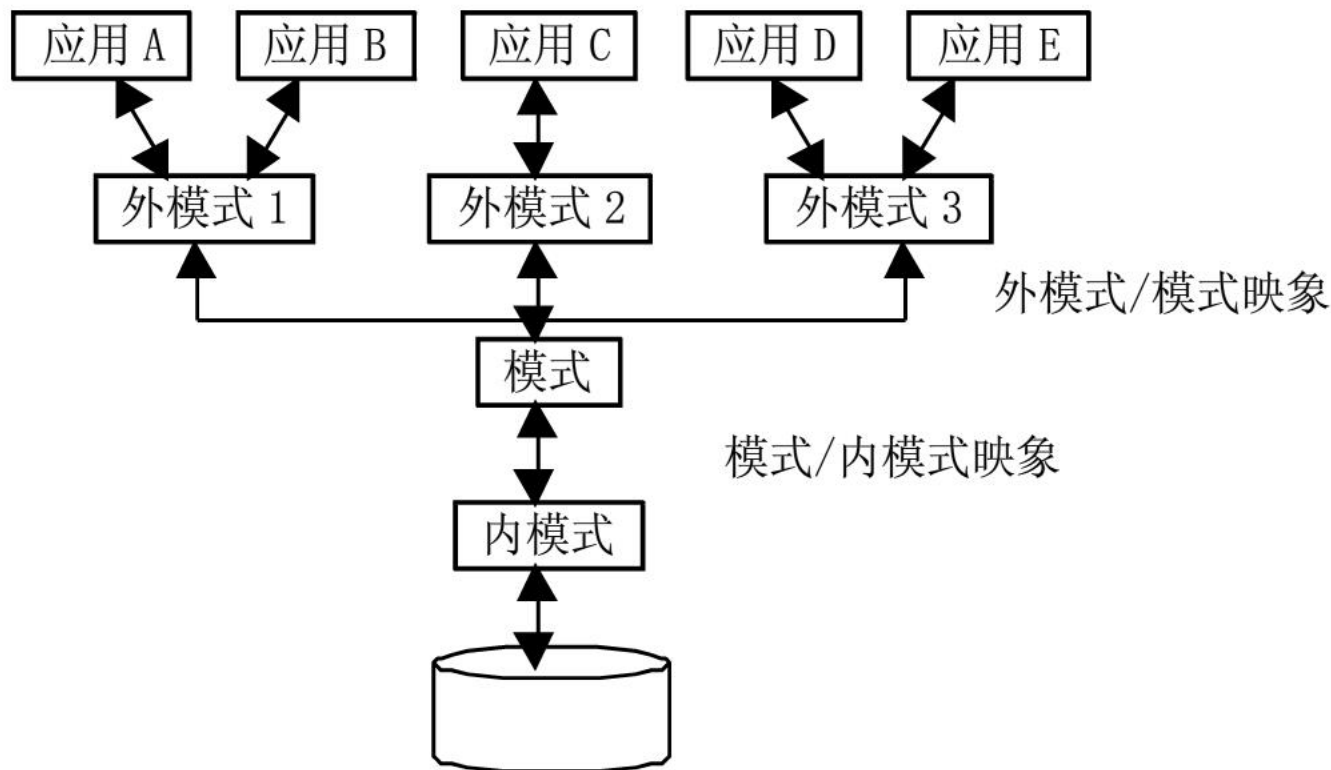
- (1) 适应客观数据和功能变化的需求
- (2) 适应计算机技术发展的需求
- (3) 完善系统
 - 处理系统测试阶段发现的问题和故障。
- (4) 运行纠错
 - 处理只能在系统运行阶段才暴露的错误。

4.2 数据库设计和编码设计

4.2.1 数据库设计



数据库系统的三级模式结构





数据库设计内容：

- 1、用户权限的设计
 - 任何用户都不能随意存取数据库中的任何记录。
- 2、索引文件的设计
 - 检索方式：
 - 顺序：适用于记录不多的情况。
 - 索引文件：采用折半查找等快速查找算法。
 - 索引特点：占空间、维护复杂
- 3、中间文件或临时文件的设计
 - 使用后删除功能的设计。
- 4、视图（View）的设计
 - 视图是虚表，只保留定义。



4.2.2 编码设计

- 1、编码设计的地位和作用
- 2、分类的原则和方法
- 3、代码设计的原则和方法
- 4、分类编码的设计步骤

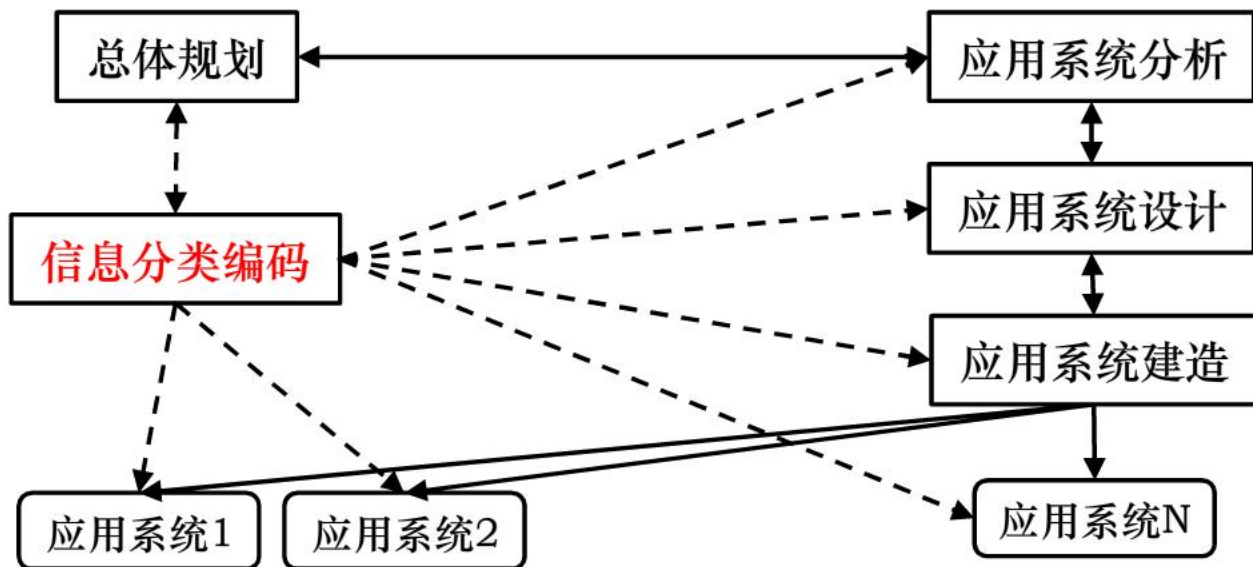
4.2.2 编码设计

1、编码设计的地位和作用

- 信息系统建设的指导思想

- 数据位于数据处理的中心，数据模型稳定，处理多变

- 没有编码设计，无法进行信息共享。

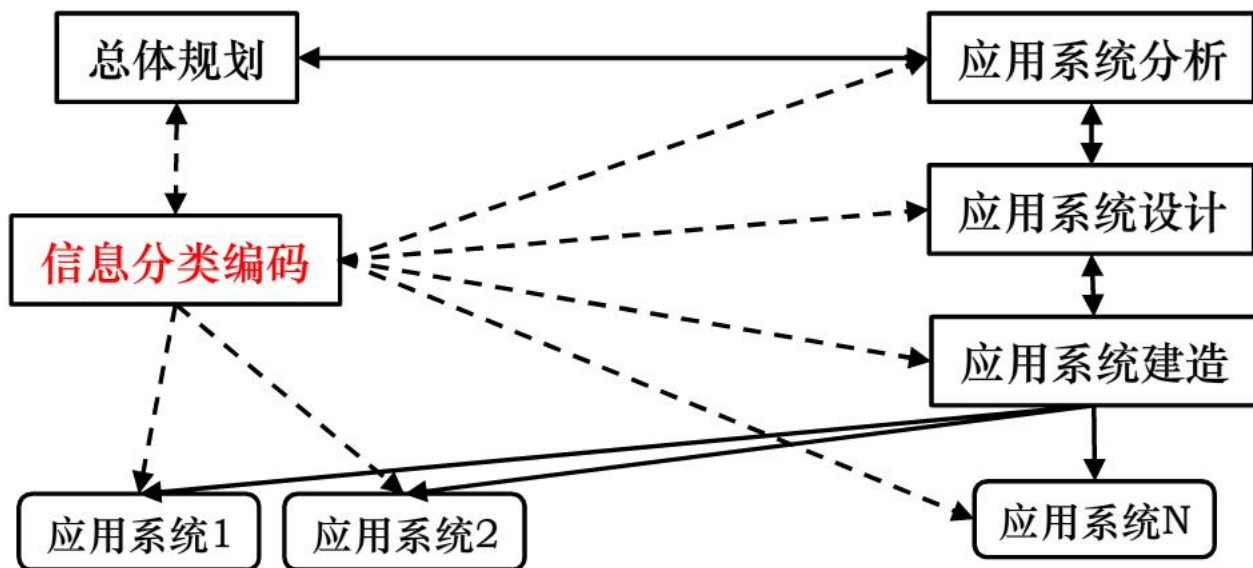


4.2.2 编码设计

1、编码设计的地位和作用

■ 编码设计步骤：

- 对编码对象进行科学分类，自上而下；
- 按照一定的规则设计代码。



2、分类的原则和方法

■ 分类原则：

■ 科学性：

- 基于分类对象中最稳定的本质属性或特征分类。

■ 系统性：

- 将按分类对象的属性特征排序，形成合理的科学分类体系。

■ 可扩展性：（有可拓展的类目）

- 保证增加新的事物或概念时，不打乱已建立的分类体系；
- 为下级系统在本分类体系的基础上进行延拓细化创造条件。

■ 兼容性：

- 与国际标准、国家标准、行业标准等协调一致。

■ 综合实用性：

- 从系统出发，把全局问题放在系统整体中处理，达到全系统最优。

2、分类的原则和方法

- **分类的基本方法：** 线分类法、面分类法。

- **(1) 线分类法(层级分类法)：**

- 将初始的**分类对象**（即被划分的事物或概念）按所选定的若干**属性或特征**（作为分类的划分基础）逐次地分成相应的若干层级的类目，并排成一个有层次的、逐级展开的分类体系。

- **术语（线分类体系中）：**

- **上位类：**

- 一个类目**相对于**由它直接划分出来的**下一级类**目而言，称为上位类。

- **下位类：**

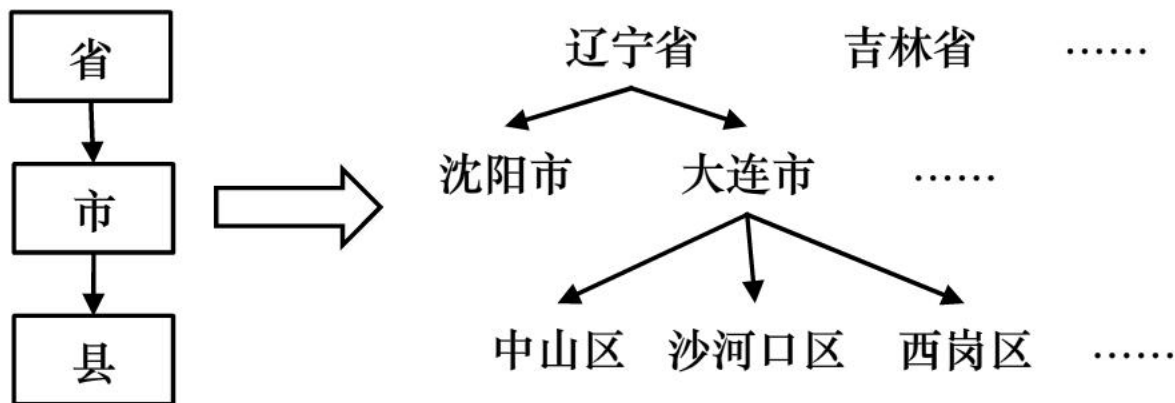
- 由上位直接划分出来的下一级类目**相对于上位类**而言，称为下位类。

- **同位类：**

- 由一个类目直接划分出来的下一级中的各类目，**彼此**称为同位类。

■ 举例

- GB2260-86 《中华人民共和国行政区划代码》采用的是线分类法，将全国行政区划分为三层：
 - 第一层为省（自治区、直辖市）
 - 第二层为地区（市、州、盟）
 - 第三层为县（市、旗、镇、区）。





■线分类法的原则：

- 上下位类目范围
 - 由某一上位类划分出来的下位类类目的总范围应与上位类类目相等；
- 划分基准的选择
 - 当某一个上位类类目划分成若干个下位类类目时，应选择一个划分基准：
- 同位类类目的关系
 - 之间不交叉、不重复，并只对应于一个上位类；
- 分类顺序
 - 要依次进行，不应有空层或加层。



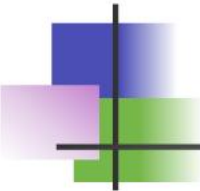
■ 线分类法的优缺点

■ 优点：

- 层次性好，较好地反映类目之间的逻辑关系
- 使用方便，既符合手工处理信息的传统习惯，又便于计算机处理信息

■ 缺点：

- 结构弹性较差，分类结构一经确定，不易改动；
- 分类层次较多时，为其所设计的代码位数会较大，影响数据处理的效率与速度。



2、分类的原则和方法

- (1) 线分类法
- (2) 面分类法
 - 按属性或特征分面
 - 将所选定的分类对象的若干个属性或特征视为若干个“面”；
 - 面中分类目
 - 每个“面”中有可分成彼此独立的若干个类目；
 - 面排列
 - 按一定的顺序将各个“面”平行排列。
 - 形成复合类目
 - 根据需要将这此“面”中的类目按指定的顺序组合在一起，形成一个新的复合类目。



■ 举例

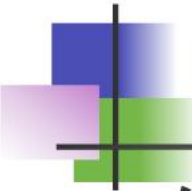
- 服装的分类采用面分类法。
- 选择服装材料、男女式样、服装款式三个属性作为三个“面”，每个“面”又可分成若干个类目，如下表。

材料	男女式样	服装款式
纯棉	男式	中山装
纯毛	女式	西服
中长纤维		连衣裙
.....		



■面分类法的原则：

- “面” 选择
 - 以分类对象**本质的属性**或**特征**作为分类对象的各个“面”
- 类目间关系
 - 不同面内的类目**不应相互交叉**，也**不能重复**出现
- “面” 位置
 - 有严格的固定位置
- “面” 的选择以及位置的确定，根据实际需要而定。



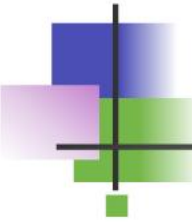
■面分类法的优缺点：

■ 优点：

- 弹性较大，一个“面”内类目改变，不会影响其它的“面”；
- 适应性强，可根据需要组成任何类目，同时也便于计算机处理信息；
- 易于添加和修改类目。

■ 缺点：

- 不能充分利用容量，可组配的类目很多，但有时实际应用的类目不多；
- 难于手工处理信息。



3、代码设计的原则和方法

(1) 代码的功能

- 信息代码：
 - 是一个或一组有序的、易于计算机和人识别与处理的符号，有时也称为“码”。
- 功能：（标识、分类、排序、特定含义）
 - 标识：鉴别编码对象的唯一标志；
 - 分类：当按编码对象的属性或特征分类，并赋予不同的类别代码时，代码又可以作为区分编码对象类别的标志；
 - 排序：当按编码对象发现（产生）的时间、所占有的空间或其他方面的顺序关系分类，并赋予不同的代码时，代码又可作为编码对象排序的标志；
 - 特定含义：代码是在一定分类体系下产生的，因此代码可提供一定的特定含义。



(2) 代码设计的基本原则

- 唯一性
 - 在一个分类编码标准中：
 - 每一个编码对象仅有一个代码，
 - 一个代码只惟一表示一个编码对象。
- 合理性
- 可扩充性
- 简单性
- 适用性
- 规范性



(2) 代码设计的基本原则

- 唯一性
- 合理性
 - 代码结构要与分类体系相适应。
- 可扩充性
- 简单性
- 适用性
- 规范性



(2) 代码设计的基本原则

- 唯一性
- 合理性
- 可扩充性
 - 留适当的后备容量
 - 以适应不断扩充的需要。
- 简单性
- 适用性
- 规范性



(2) 代码设计的基本原则

- 唯一性
 - 合理性
 - 可扩充性
 - 简单性
 - 适用性
 - 规范性
- 代码结构应简单：
 - 节省存储空间；
 - 减少代码差错率；
 - 提高处理计算机效率。



(2) 代码设计的基本原则

- 唯一性
- 合理性
- 可扩充性
- 简单性
- 适用性
- 规范性

➤ 代码要反映编码对象的特点：
➤ 有助记忆、填写。



(2) 代码设计的基本原则

- 唯一性
- 合理性
- 可扩充性
- 简单性
- 适用性
- **规范性**
 - 在一个分类编码标准中，代码：
 - 类型、结构以及编写格式必须统一。



(3) 常用代码设计方案

- 顺序码
- 表意码
- 位别码
- 合成码



(3) 常用代码设计方案

■ 顺序码：

- 一种用连续数字代表项目名的编码。

- 例如：

- 00 学校
- 01 数学系
- 02 物理系
-

- 特点：

- 代码位数少；
- 一个项目一个连续号；
- 处理容易；
- 设计和管理较容易。



(3) 常用代码设计方案

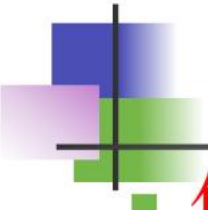
■ 表意码

- 把直接或间接表示编码化对象属性的某些文字、数字、记号**原封不动**作为**编码**。

编码	中文	英文
MT	米	Meter
CM	厘米	Conti-meter
MM	毫米	Milli-meter
TN	吨	Ton
KG	公斤	Kilo-guam
LR	克	Litre
.....		

■ 特点：

- 见码知意，易记、易理解
- 编码数量增加时，位数据增加，使处理不变



(3) 常用代码设计方案

■ 位别码：

- 用不同的位来代表不同的类别，每一位具有某一类别含义。

■ 例如：

- 在行政区划代码（GB2260—84）中：
 - 第一、二位描述省（自治区、直辖市）、
 - 第三、四位描述市（市、州、盟）、
 - 第五、六位描述县（市、旗、镇、区）

1	2	3	4	5	6
省		市		区	



(3) 常用代码设计方案

■ 例如北京市：

- | | |
|---------------|---------------|
| ■ 110000 北京市 | ■ 110109 门头沟区 |
| ■ 110100 市辖区 | ■ 110111 房山区 |
| ■ 110101 东城区 | ■ 110112 通州区 |
| ■ 110102 西城区 | ■ 110113 顺义区 |
| ■ 110103 崇文区 | ■ 110114 昌平区 |
| ■ 110229 延庆县 | ■ 110115 大兴区 |
| ■ 110104 宣武区 | ■ 110116 怀柔区 |
| ■ 110105 朝阳区 | ■ 110117 平谷区 |
| ■ 110106 丰台区 | ■ 110200 县 |
| ■ 110107 石景山区 | ■ 110228 密云县 |
| ■ 110108 海淀区 | ■ 110229 延庆县 |



(3) 常用代码设计方案

■ 合成码：

- 把编码对象用两种以上的代码方案进行组合。

- 例1：

代码	名称
.....	
210000	辽宁省
210100	沈阳市
210200	大连市
210201	大连市中山区
210202	大连市沙河口区
.....	

- 编码方案：

- 省、市、县用顺序码，
- 各位用位别码。
- 位别码和顺序码相结合。

(3) 常用代码设计方案

- 例2：位别码：描述分类属性；顺序码：每类中用

材料		男女式样		服装款式	
01	纯棉	01	男式	001	中山装
02	纯毛	02	女式	002	西服
03	中长纤维			003	连衣裙
.....					

- 服装代码：
 - “0101001” 表示一款纯棉男式中山装
 - “0102002” 表示一款纯棉女式西服
- 特点：
 - 容易分类、增加编码层次；
 - 可从多方面去识别，做各种分类统计容易；
 - 位数和数据项个数较多。



4、分类编码的设计步骤

- ①选定编码化的对象
 - 确定对象所属子系统，即：需编码的项目，编码的名称
- ②明确编码化的目的及编码在系统中的作用
- ③确定编码使用范围
 - 应适用于一个企业中各部门，还可在外单位使用
- ④确定编码使用期限
 - 无特殊情况，应永久性使用
- ⑤决定编码方法
 - 合适的分类方法和代码种类及校验方式

5、编码设计实例

- 例：教学管理系统，确定编码对象如表4-5。
- 课程编码设计方案：
 - 5位数字组成

编码对象	使用范围	使用期限	建议使用的编码方法
学生学号	整个系统	长期	合成码
课程编码	整个系统	长期	合成码
教师编号	整个系统	长期	合成码
教室编码	整个系统	长期	合成码

位数	含义	取值及含义
1	课程大类号	0: 人文社科、体育、外语； 1: 经济、管理学科； 2: 自然学科； 3: 工程与技术；
2、3	课程中类号	大类号 0: 01政治； 02哲学； 03文学； 04法律； 05历史； ... 大类号 1: 01经济； 02 管理。 大类号 2: 01数学； 02物理（包括实验物理）； 03化学。 大类号 3: 01机械工程； 02电气技术； 03电子技术与信息技术； ... 大类号 4: 01毕业设计； 02课程设计； 03各类实习； ...； 05军训
4、5	某一类下的课程顺序号	00101: 政治经济学； 10101: 计量经济学 20101: 高等数学； 20102: 线性代数； 40501: 军训...



4.3 人机界面设计

- 4.3.1 输入/输出方式
- 4.3.2 操作模式的设计
- 4.3.3 界面统一性设计



4.3.1 输入/输出方式

- 输入/输出方式
- 校对方式



4.3.1 输入/输出方式

■ 输入/输出方式

- ① 键盘输入
- ② A/D, D/A (即模→数、数→模) 转换
- ③ 网络或通讯传输
- ④ 报表
- ⑤ 图形输出
- ⑥ 批处理/实时处理



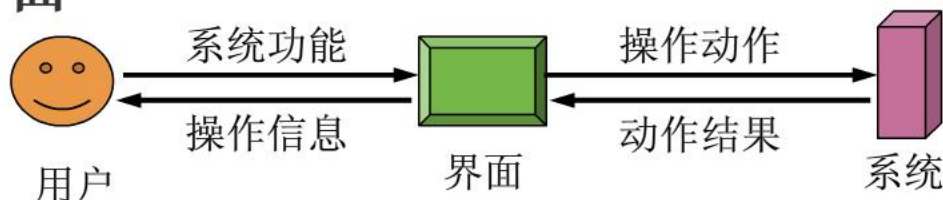
4.3.1 输入/输出方式

- 输入/输出方式
- 校对方式
 - ① 人工校对
 - ② 二次键入
 - ③ 利用统计中的平衡关系或财务帐目中的平衡关系进行校对

入学年份	本科生人数	大专生人数	学生总数
1993	1589	189	1778
1994	1487	127	1614
1995	2001	193	2194
1996	1983	270	2253
合计	7060	779	7839

4.3.2 操作模式的设计

■ 1. 界面



■ 界面：系统的**感知器官**。

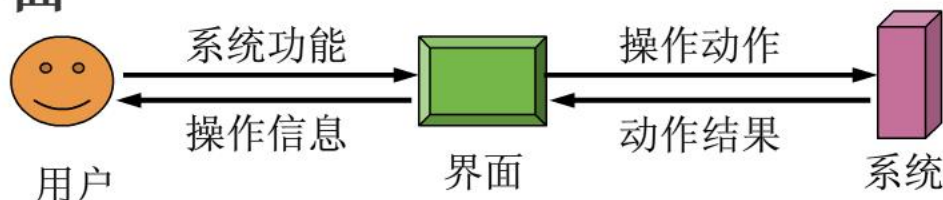
■ 界面的特点：**可见性、可供性**。

■ **可见性：**

- 有充分的提示信息，说明哪些是用于操作的以及它们的操作方法，提示用户如何实现人机交互。

4.3.2 操作模式的设计

■ 1. 界面



■ 界面：系统的**感知器官**。

■ 界面的特点：**可见性、可供性**。

■ **可供性：**

■ 每一项功能及其操作方法对于界面所处环境中的大多数用户来说是显而易见的，只需看一眼就可以有所了解。

■ 可供性的作用：提供了大量的与操作有关的线索。



4.3.2 操作模式的设计

■ 2.元动作

■ 含义:

- 是用户实现各种操作最基本的单元。

■ 作用:

- 可以实现在元动作的集合上的所有人机交互。

■ 设计方法:

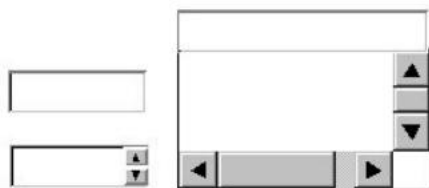
- 自顶向下。

- 例如：输入日期可分解为元动作：年、月、日。

4.3.2 操作模式的设计

■ 3.操作模式

- 属性值的键盘输入：
 - 文本输入
 - 下拉菜单
 - 表格选择



- 功能值的实现方法：
 - 功能按钮
 - 功能菜单
 - 图标按钮

输入学生信息

输入学生信息 修改学生信息



输入学生信息

- 等待系统处理的形式：
 - 进度条
 - 漏斗

- 属性值的A/D输入：
 - 条形码



4.3.2 操作模式的设计

- 4.步骤
 - 分解元动作
 - 进行操作模式的设计
 - 进行界面的统一性设计（后面讲）



4.3.2 操作模式的设计

■ 5. 示例

- 学生基本信息表管理，采用屏幕——键盘交互方式。
- 实现的功能：
 - 输入新生信息、修改错误信息、修正学籍变动信息、删除退学学生信息、删除已毕业学生信息、查询信息。

■ 分析

- (1) 查询功能所需的属性值：
 - 学号、姓名、班级、出生日期、专业、年级、学院。
- (2) 分析查询条件的数据类型，分解出元动作集合：
 - 学号、姓名、班级、专业、学院；
 - 出生日期：分解为年、月、日。
- (3) 分解功能性动作：
 - 输入学生信息、修改学生信息、删除学生信息、查询信息

4.3.2 操作模式的设计

5. 示例

- 学生基本信息 方案一:
- 实现的功能 方案二:
- 输入新生学生信息 方案三:







■ 设计

- (1) 设计属性值的操作方案:
 - 学号、姓名、班级: 键盘输入, 可有可无, 文本方式输入。
 - 出生日期: 键盘输入, 年、月、日。 年 月 日
 - 专业、学院: 键盘输入。 学院 专业
- (2) 设计功能性动作方案:
 - 输入学生信息、修改学生信息、删除学生信息、查询信息:
 - 功能按钮、功能菜单、图标按钮。



4.3.2 操作模式的设计

- 6. 注意事项

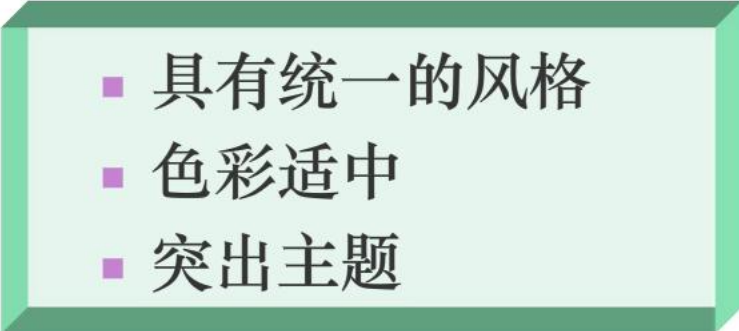
- 操作要尽可能地：

- ① 发挥计算机的优势；
- ② 符合用户的操作习惯；
- ③ 简单、明了，使用方便。



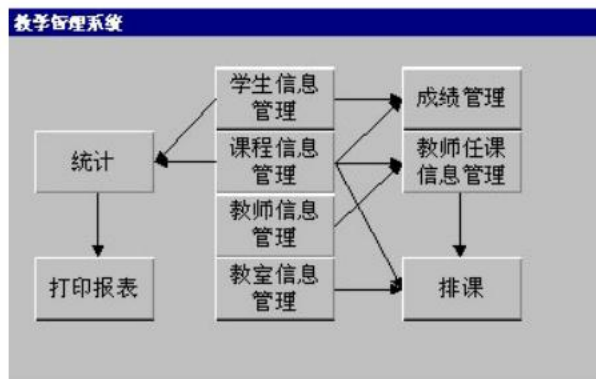
4.3.2 界面统一性设计

- 1.风格设计
- 2.导航设计
- 3.可靠性设计

- 
- 具有统一的风格
 - 色彩适中
 - 突出主题

4.3.2 界面统一性设计

- 1. 风格设计
- 2. 导航设计
- 3. 可靠性设计



(a) 带有工作流程的菜单设计方案

教学管理系统								
学生信息管理	课程信息管理	成绩信息管理	教师信息管理	教室信息管理	教师任课管理	排课	统计	打印报表
基本信息管理		考试成绩		计算机排课			开课统计	
奖励信息管理		重修成绩		排课调整			班级人数统计	
处分信息管理				打印班级课表			异动信息统计	
异动信息管理				打印教师课表				
课外活动管理				打印教室课表				

(b) 经常使用的菜单设计方案

4.3.2 界面统一性设计

- 1. 风格设计
- 2. 导航设计
- 3. 可靠性设计

- 良好的容错性
- 充分的提示信息
- 光标当前位置
- 运行状态

货物重量

(a)

货物重量 (kg)

(b)

■ 减少操作错误的措施：

- 输入操作符尽可能简单、易记忆，提示要简短、明了；
- 对一些参数设置缺省值；
- 容忍用户操作失误；
- 有运行状态提示，防止错误积累；
- 检测用户错误，屏蔽输入错误，及时提供出错信息。



4.3.2 界面统一性设计

- 操作模式、界面风格统一的目标：
 - ① 简单性
 - ② 可靠性
 - ③ 易学习与易使用性
 - ④ 立即反馈性

4.4 功能设计的基本方法

■ 原则

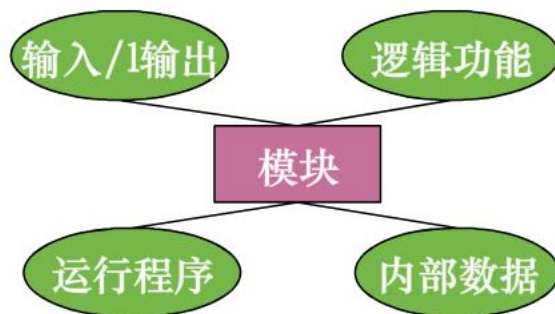
- 自顶向下。

■ 方法

- 视系统或其组成部分为暗盒，基于输入与输出及对数据变换功能的**模块**的逐层分解。

■ 模块

- 具有4种属性的一组程序语句。
- 4种属性：



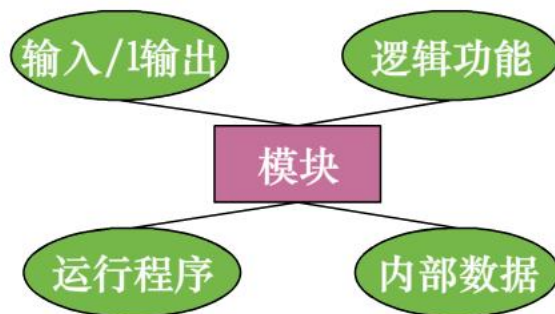
4.4 功能设计的基本方法

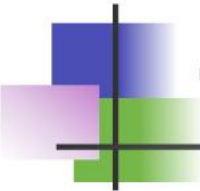
■ 示例

- 计算“圆的面积”模块。

■ 4种属性：

- 输入：R（半径）；输出：S（面积）。
- 逻辑功能：
 - 从调用者处获得R；
 - 将产生的S返回给调用者。
- 内部数据：
 - π （圆周率）。
- 运行程序：
 - 计算圆面积（S）程序。





4.4 功能设计的基本方法

- 表示

- 结构图。

- 步骤

- (1)将逻辑结构（数据流程图）转换成初始的结构图；
 - (2) 对初始结构图进行优化设计。

4.4.1 结构图

■ 符号：

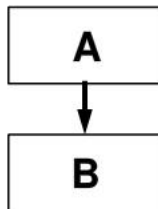
- 模块
- 模块之间的调用关系
- 数据传递关系

■ 数据：

- 一个数据元素
- 一组数据元素构成的数据结构

■ 标识：

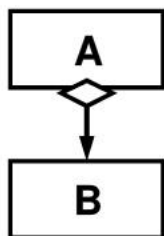
- 一些关于处理结果的标志（返回标志）。
- 例如：没找到相应记录的标志、文件结束标志等。



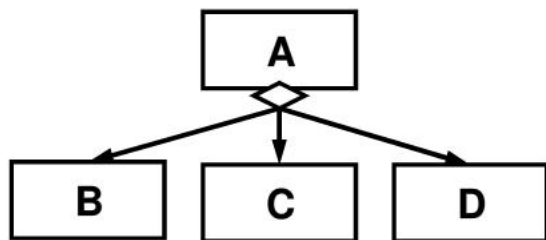
4.4.1 结构图

符号:

- 判断调用关系



(a) 满足某一个条件值时的调用



(b) 满足条件中三个不同的取值时的调用

(a) 在A模块中有语句结构:

如果 条件1 成立

则 调用B

(B) 在A模块中有语句结构:

如果 条件取值为1 则 调用B

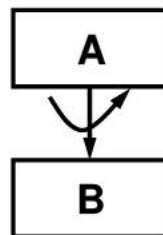
如果 条件取值为2 则 调用C

如果 条件取值为3 则 调用D

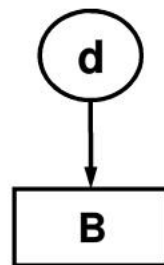
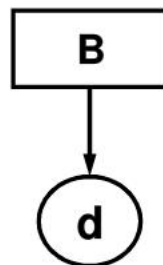
4.4.1 结构图

■ 符号：

- 循环调用关系



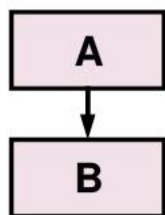
- 转接符号



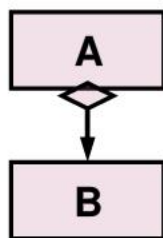
4.4.1 结构图

■ 符号：

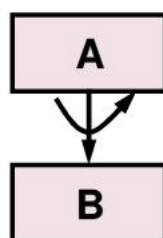
■ 调用



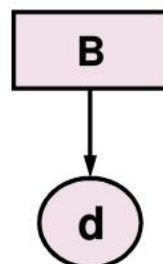
■ 判断调用



■ 循环调用



■ 转接



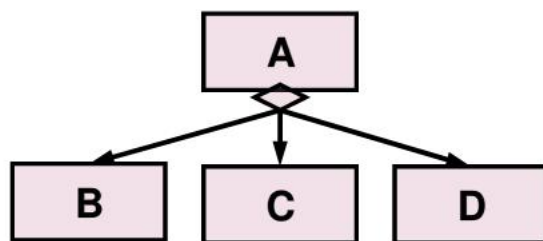
(a) 满足某一个条件值时的调用



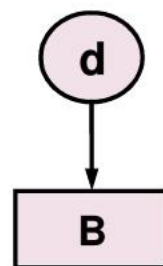
数据



标识



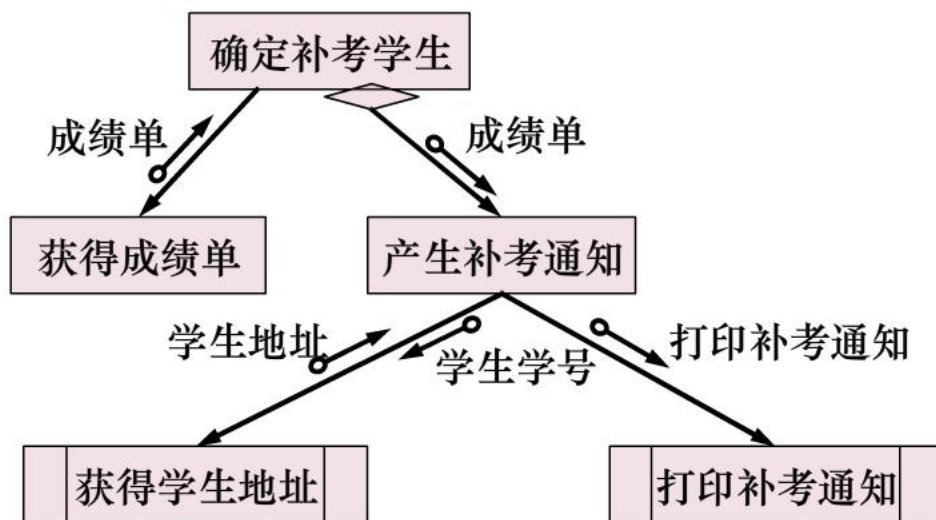
(b) 满足条件中三个不同取值时的调用



4.4.1 结构图

■ 示例：

- 根据期末考试成绩发补考通知单的结构图如下。



发补考通知单的结构图



4.4.1 结构图

■ 优点：

- 1、很高的可读性，便于理解和讨论。
- 2、“自顶向下”的原则。
- 3、具有较高的严密性、灵活性。
- 4、结构图是后期各阶段工作的重要保证。



4.4.1 结构图

- 优点：
 - 1、很高的可读性，便于理解和讨论
 - 2、“自顶向下”的原则
 - 3、具有较高的严密性、灵活性。
 - 4、结构图是后期各阶段工作的重要保证。
- 逐层扩展得到系统总体结构图。
 - 分散整个设计任务
 - 由若干个设计人员进行不同部分的设计工作；
 - 既便于管理，又能够加快设计进度，提高工作效率。



4.4.1 结构图

- 优点：
 - 1、很高的可读性，便于理解和讨论。
 - 2、“自顶向下”的原则。
 - 3、具有较高的严密性、灵活性。
 - 4、结构图是后期各阶段工作的重要保证。
- 能准确地表达系统中各个组成部分以及它们之间的联结关系；
 - 能随着数据流程图的改变而改变，灵活性强，易于修改和维护。



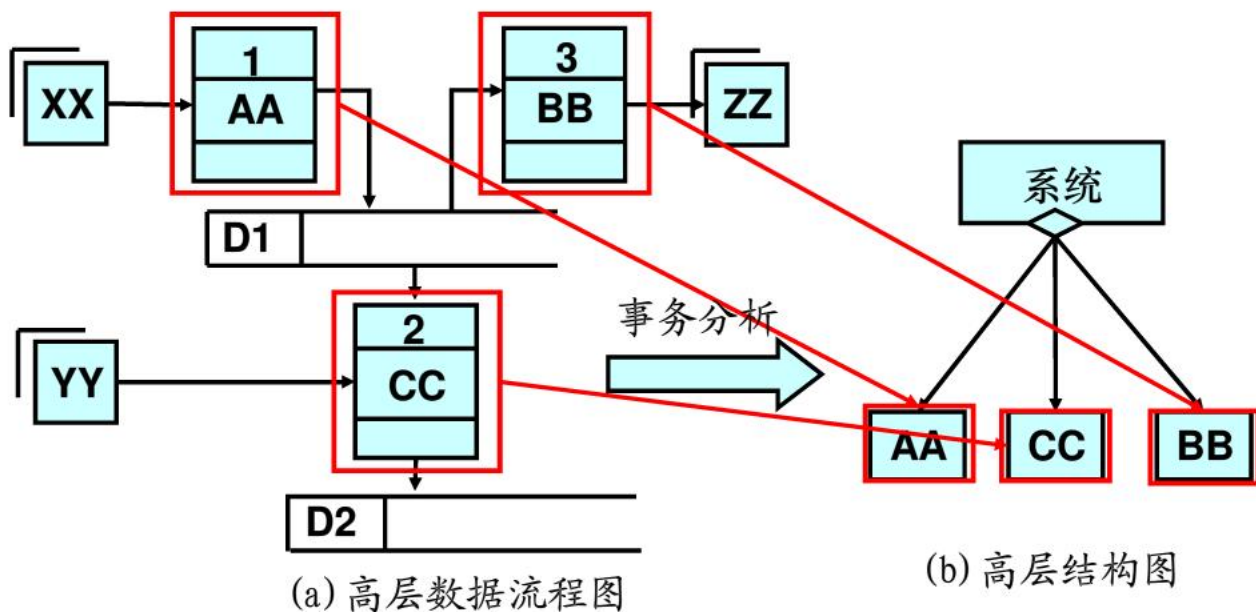
4.4.1 结构图

- 优点：
 - 1、很高的可读性，便于理解和讨论。
 - 2、“自顶向下”的原则。
 - 3、具有较高的严密性、灵活性。
 - **4、结构图是后期各阶段工作的重要保证。**
- 是系统设计的重要工具
 - 是程序设计、系统测试、系统维护工作顺利进行的重要保证。

4.4.2 功能模块转换策略

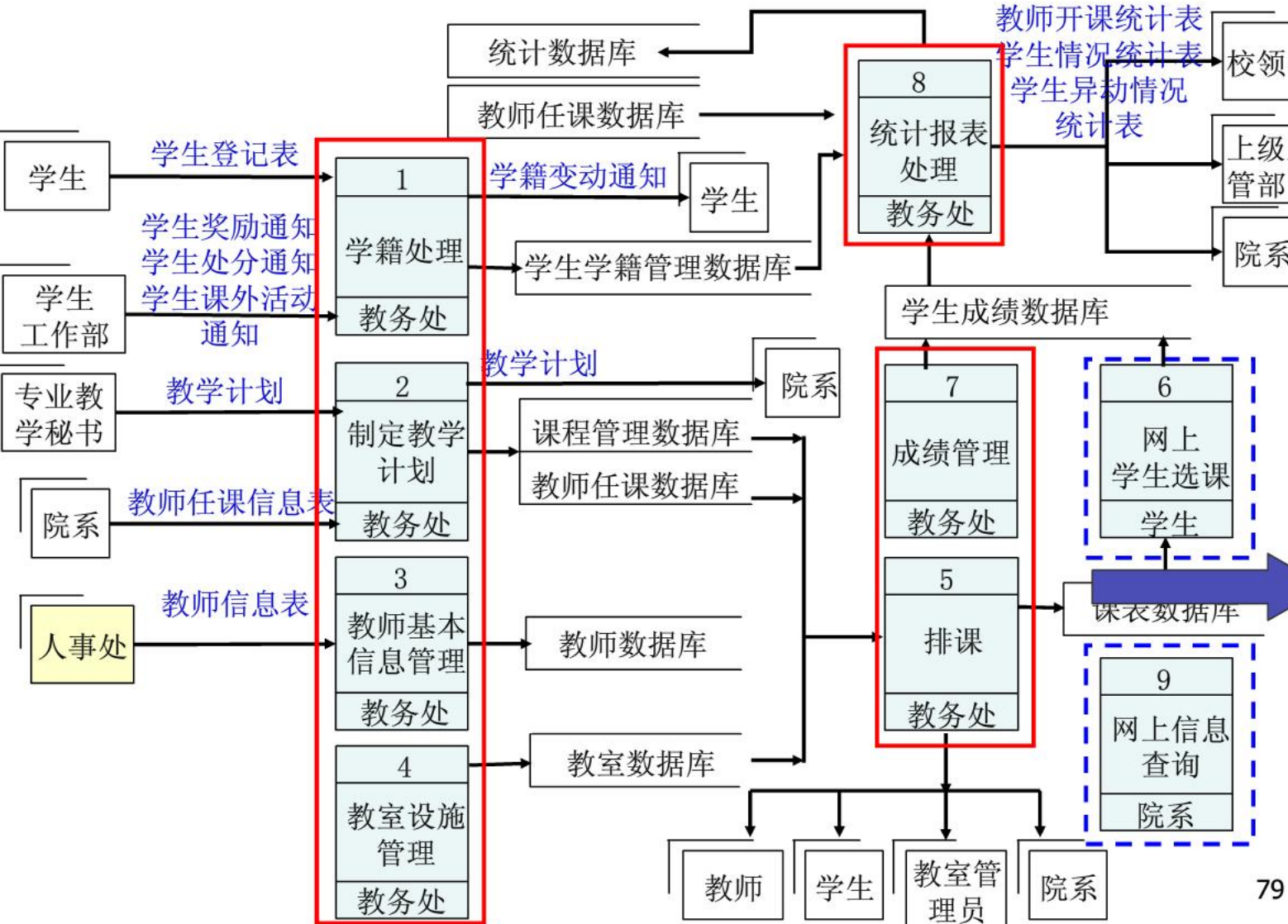
1. 事务分析

- 高层数据流图特别是一级数据流图所表示的处理功能都是相对独立的，可把它们看成是一类类**特定的事务**，将其**转换**为**结构图**。

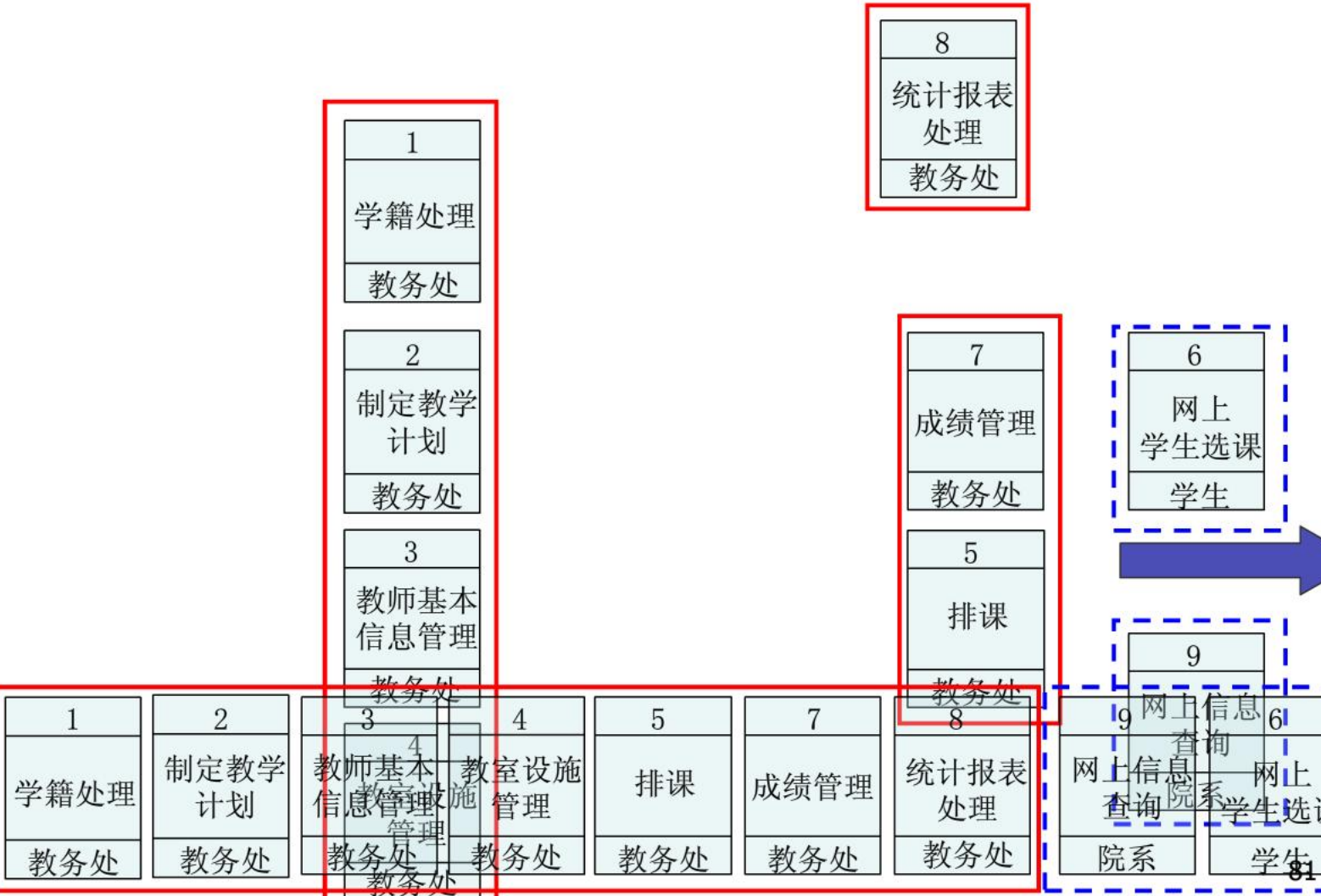


由事务分析产生的结构图

例:将教学管理系统二级数据流图转换成结构图 4.4 功能设计的基本方法

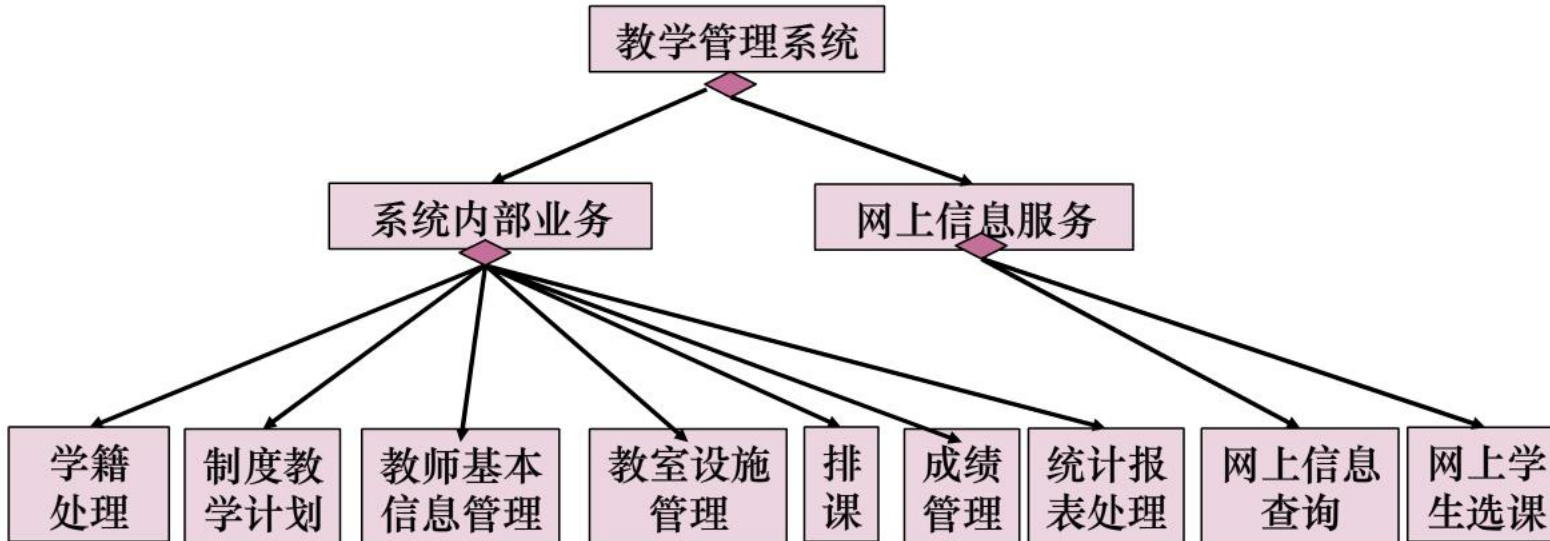


例:将教学管理系统二级数据流图转换成结构图 4.4 功能设计的基本方法

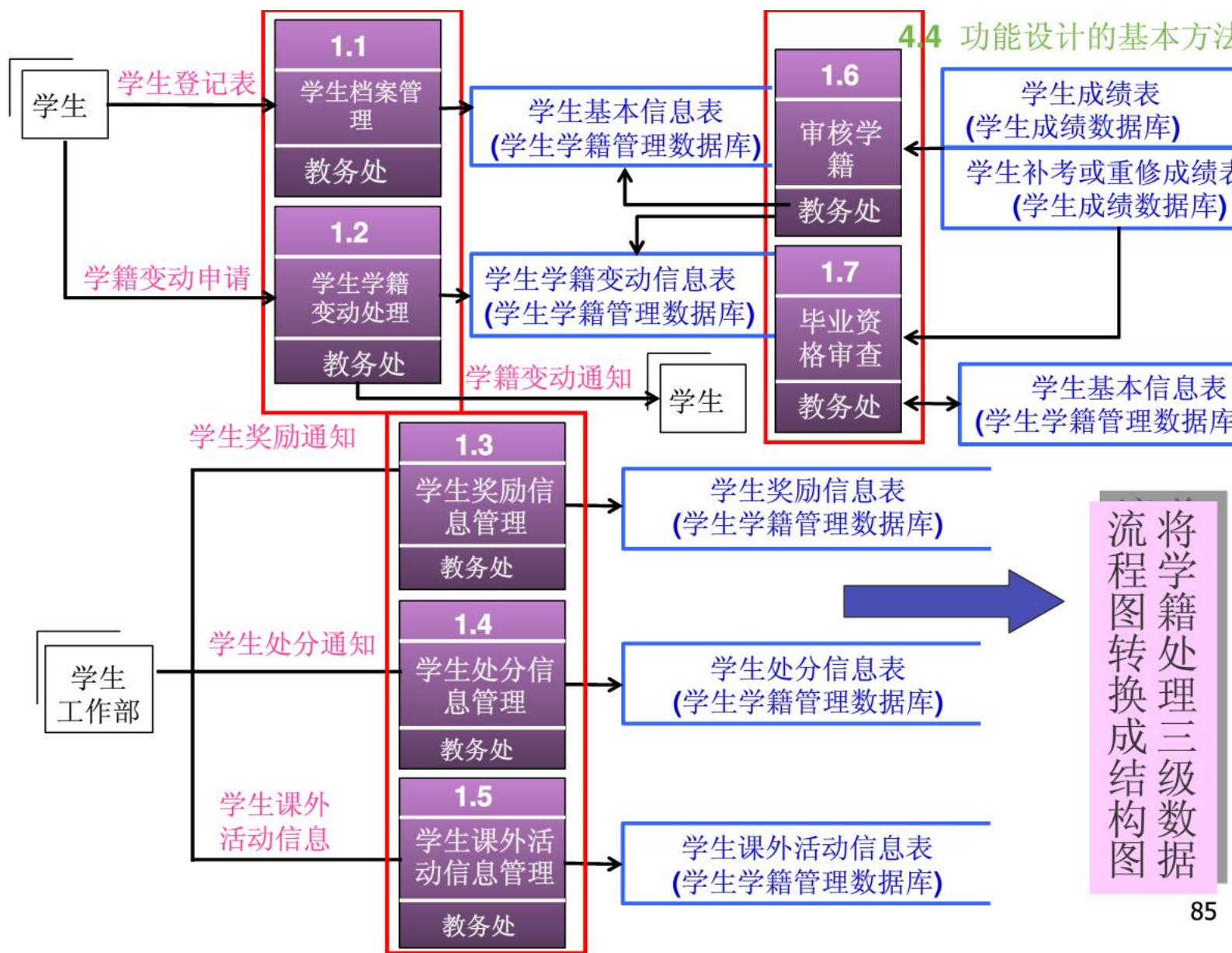


例:将教学管理系统二级数据流图转换成结构图 4.4 功能设计的基本方法

教学管理系统高层结构图



1	2	3	4	5	7	8	9	6
学籍处理	制定教学计划	教师基本信息管理	教室设施管理	排课	成绩管理	统计报表处理	网上信息查询	网上学生选课
教务处	教务处	教务处	教务处	教务处	教务处	教务处	院系	学生

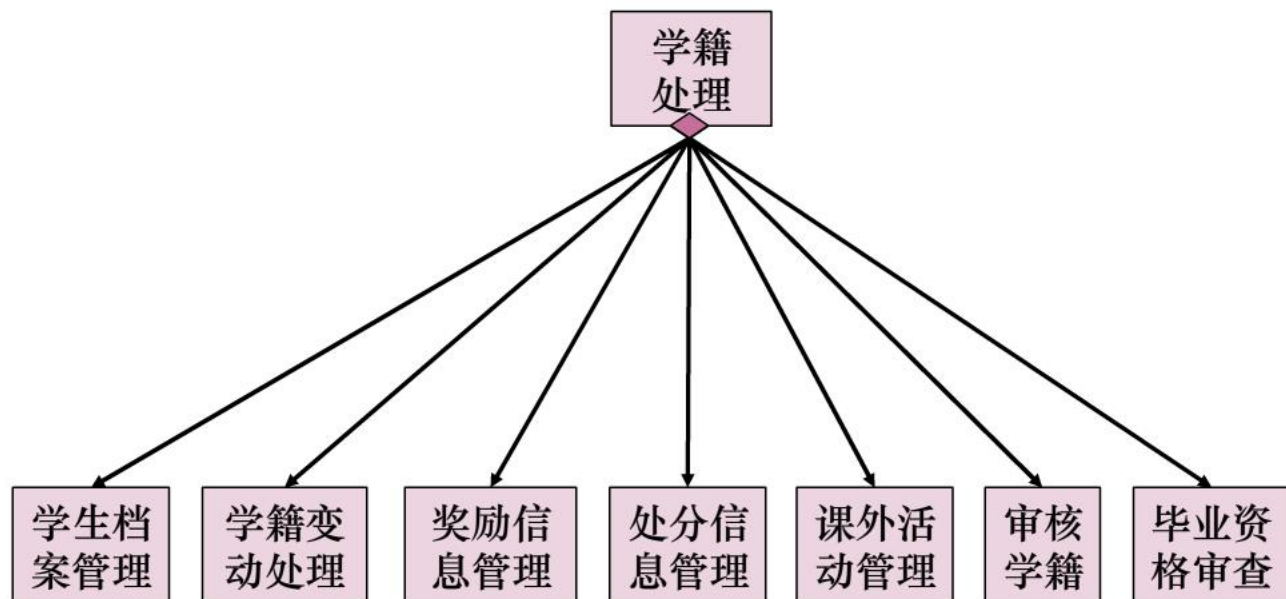


4.4 功能设计的基本方法

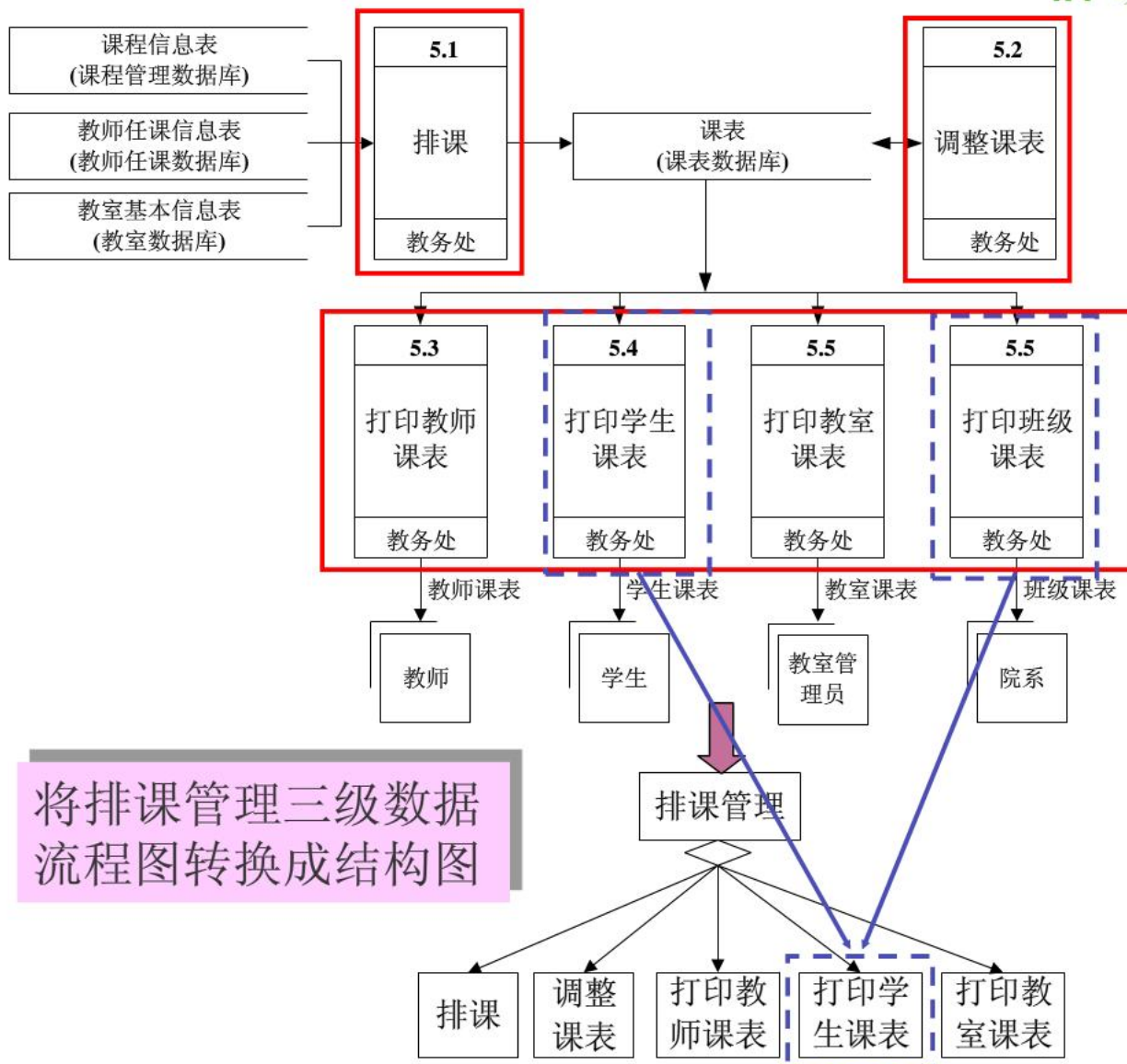


将学籍处理三级数据流程图转换成结构图

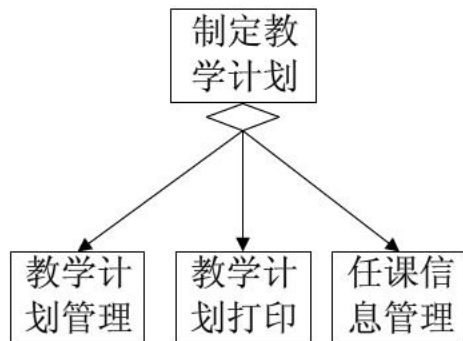
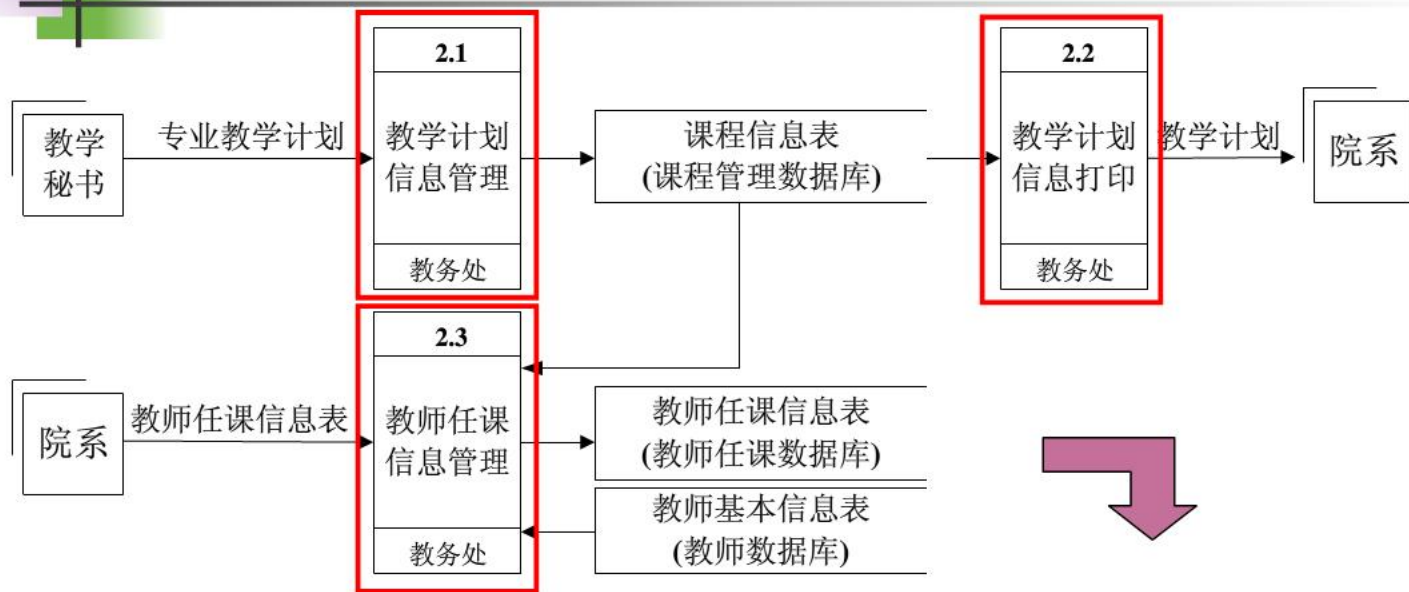
教籍处理结构图



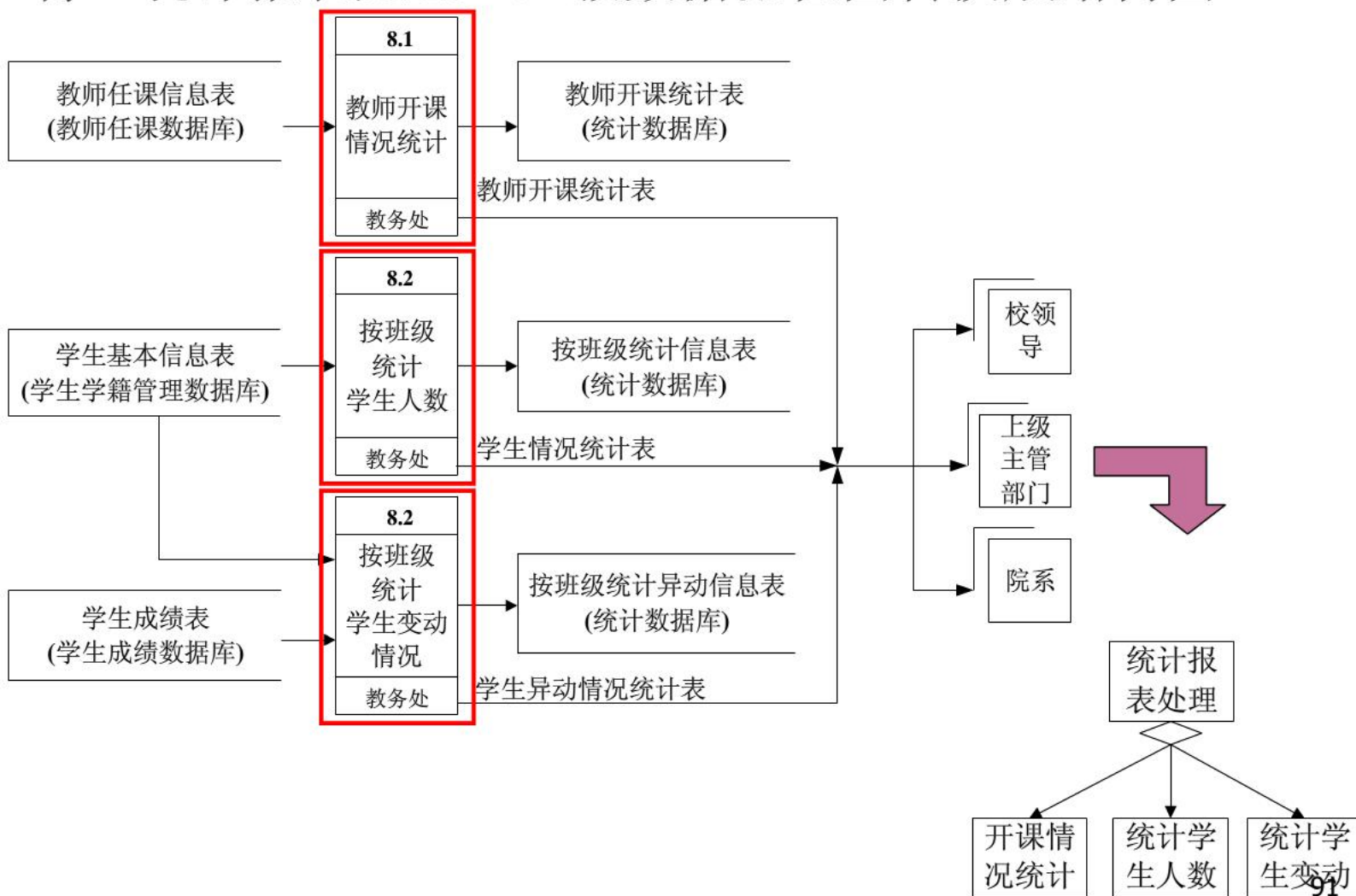
1.1	1.2	1.3	1.4	1.5	1.6	1.7
学生档案管 理	学生学籍 变动处理	学生奖励信 息管理	学生处分信 息管理	学生课外活 动信息管理	审核学 籍	毕业资 格审查
教务处	教务处	教务处	教务处	教务处	教务处	教务处



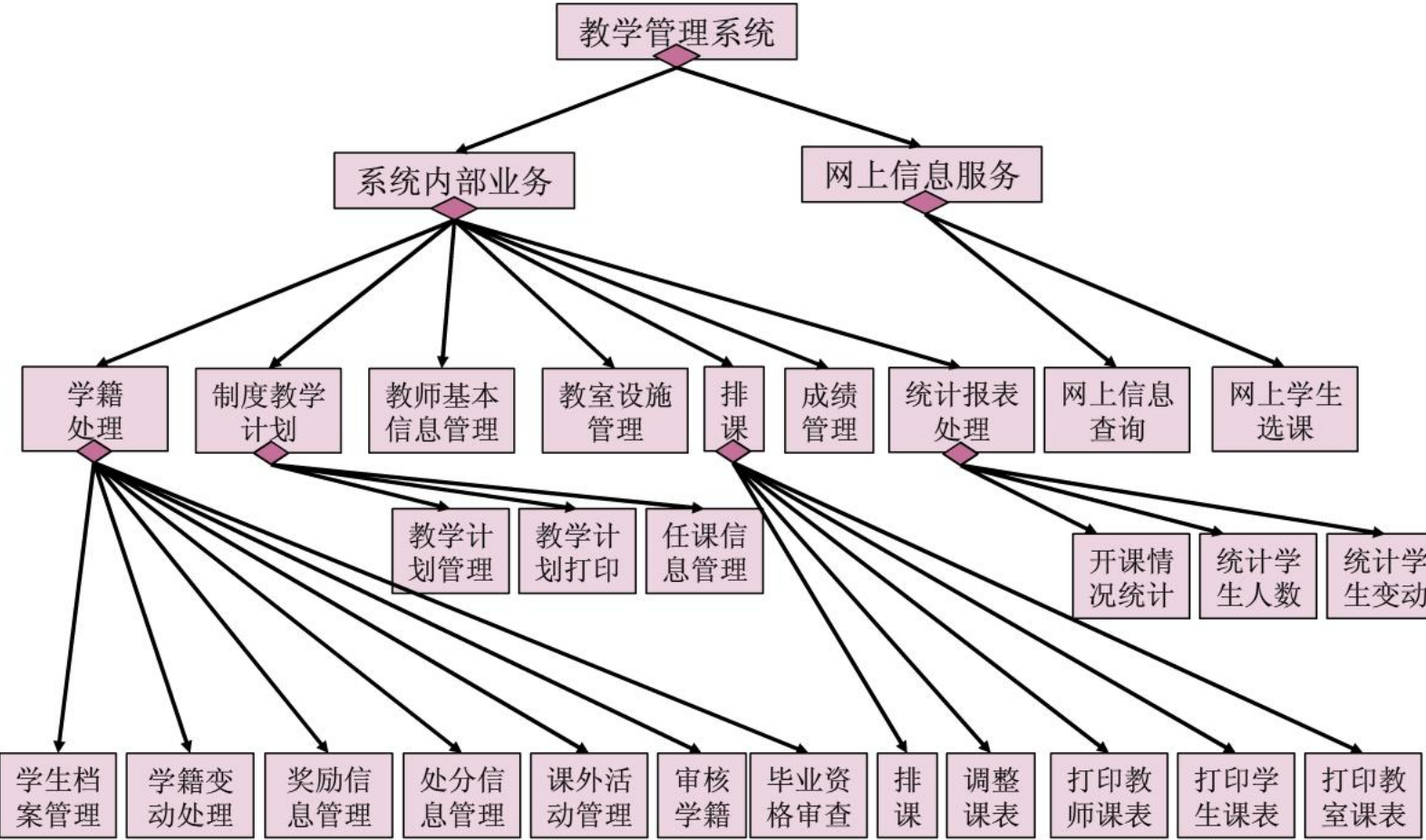
将“制定教学计划”三级数据流图转换成结构图

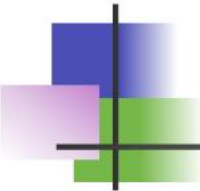


将“统计报表处理”三级数据流程图转换成结构图



扩展后的结构图





4.4.2 功能模块转换策略

■ 2. 变换分析

- 是一种把叶子层模块转换成结构图的方法。
- 处理功能的分类
 - 第一类：输入功能。
 - 第二类：变换功能。
 - 第三类：输出功能。
- 特点
 - 对数据的变换只是一些形式上的加工，如编辑、验证、排列等。
 - 假设输入数据流是X，该处理功能为G，它产生的数据流是X' ；
 - 则 $X' = G(X)$ 。



4.4.2 功能模块转换策略

■ 2. 变换分析

- 是一种把叶子层模块转换成结构图的方法。
- 处理功能的分类
 - 第一类：输入功能。
 - 第二类：变换功能。
 - 第三类：输出功能。

■ 特点

- 对数据流 X' 进行实质性的变换，即真正的处理；
- 假设它的功能为 F ，它所产生的数据流是 Y' ；
 - 则 $Y' = F(X')$ 。



4.4.2 功能模块转换策略

■ 2. 变换分析

- 是一种把叶子层模块转换成结构图的方法。
- 处理功能的分类
 - 第一类：输入功能。
 - 第二类：变换功能。
 - 第三类：输出功能。

■ 特点

- 它对输入数据流 Y' 进行形式上的加工，如排列显示格式等等；
- 假设它的功能是 H ，输出数据流 Y ，
则 $Y = H(Y')$ 。



4.4.2 功能模块转换策略

■ 2. 变换分析

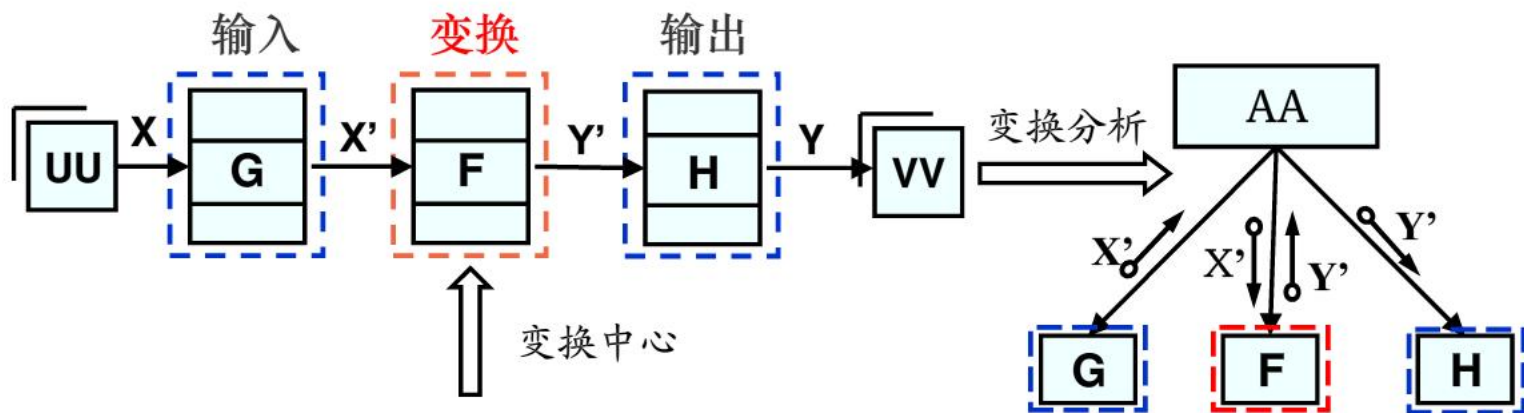
- 是一种把叶子层模块转换成结构图的方法。
- 处理功能的分类
 - 第一类：输入功能。
 - 第二类：变换功能。
 - 第三类：输出功能。
- 步骤
 - (1)对叶子层模块中的处理功能进行分类；
 - (2)确定变换中心（第二类处理功能），画出结构图。

4.4.2 功能模块转换策略

2. 变换分析

示意图

- 采用变换分析方法进行模块结构设计的示意图：



(a) 叶子层数据流程图

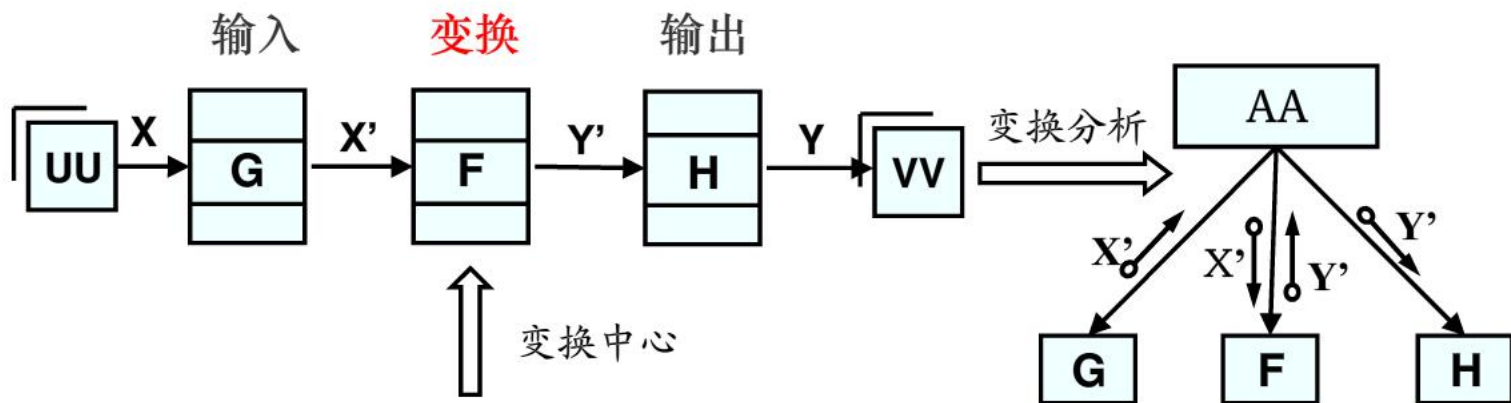
(b) 叶子层结构图

4.4.2 功能模块转换策略

2. 变换分析

示意图

- 采用变换分析方法进行模块结构设计的示意图：

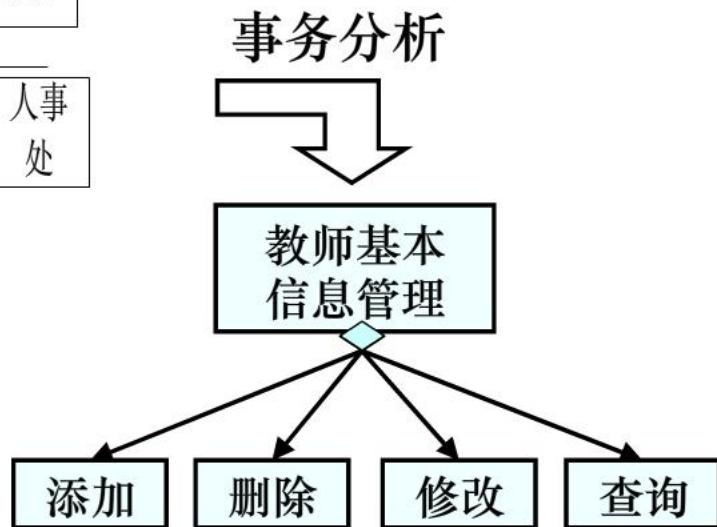
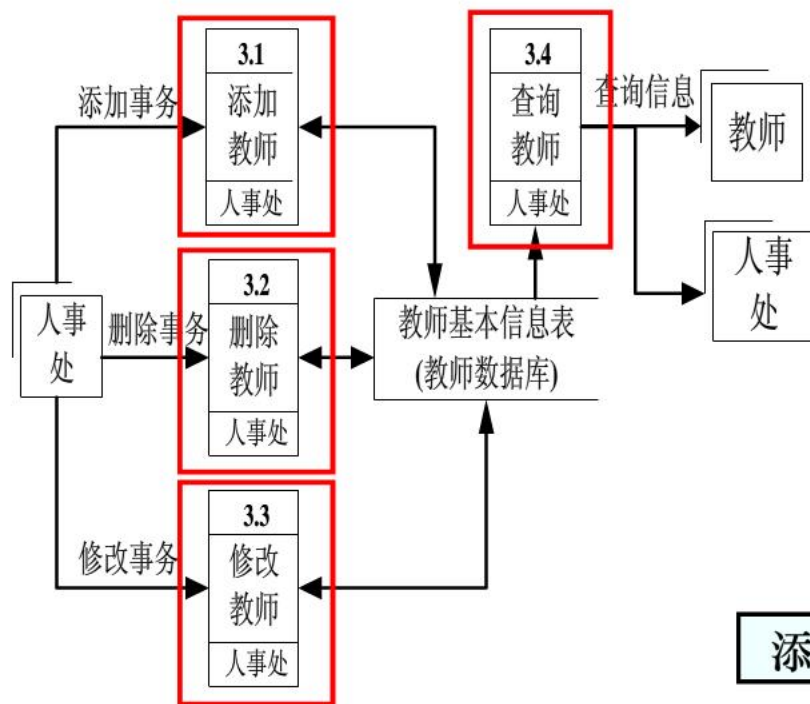


(a) 叶子层数据流程图

(b) 叶子层结构图

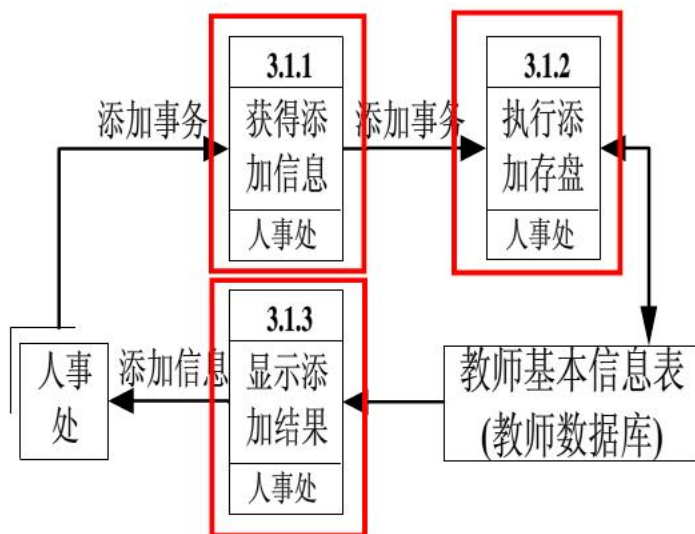
实例：绘制“教师基本信息管理”结构图

- 利用**事务分析**实现
 - 一级数据流图**转换**为一级功能结构图

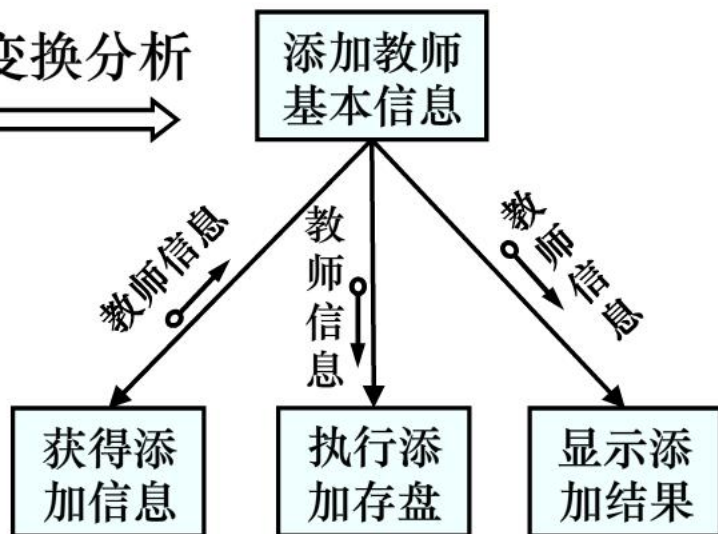
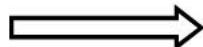


实例：绘制“教师基本信息管理”结构图

- 利用变换分析实现
 - 二级数据流图的转换
 - 添加教师基本信息 → 二级功能结构图

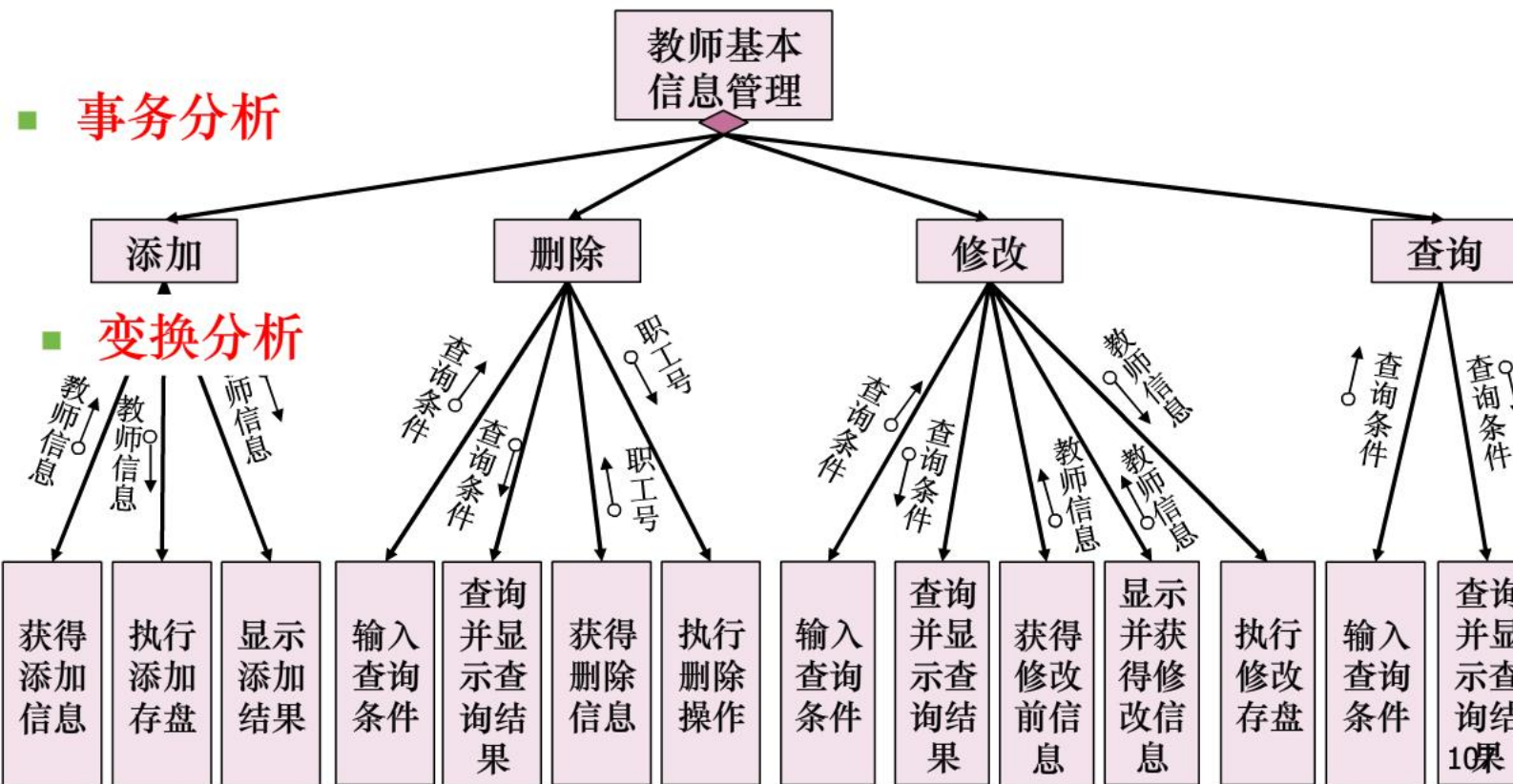


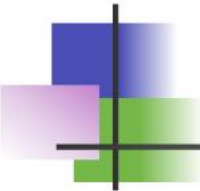
变换分析



“教师基本信息管理”结构图

■ 变换分析





4.4.3 功能模块优化设计原则

- 1. 模块之间的**耦合**原则
- 2. 模块的**内聚**性原则
- 3. 模块的**分解**原则
- 4. 模块的**扇入和扇出**原则

4.4.3 功能模块优化设计原则

■ 1. 模块之间的耦合原则

■ 耦合：

- 是对一个软件结构内不同模块之间互连程度的度量。

■ 影响耦合的主要因素：

- 模块之间信息传递的复杂性。

■ 耦合分类：

- 简单耦合、数据耦合；
- 控制耦合、特征耦合；
- 公共环境耦合；
- 内容耦合。

低

中

高



■ 简单耦合：

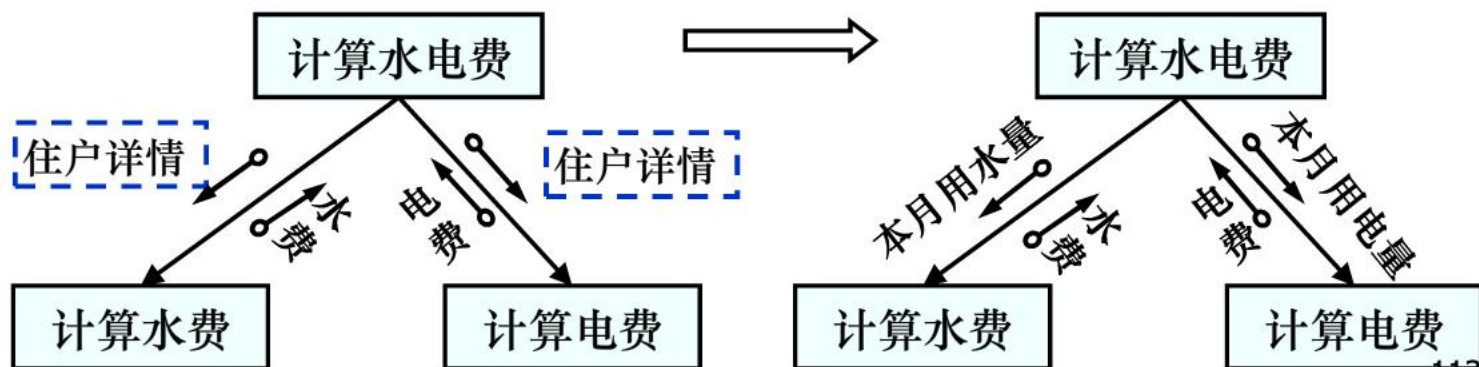
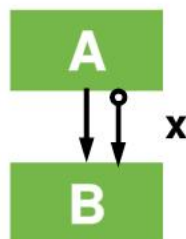
- 模块之间仅存在调用与被调用关系，而没有任何的信息传递。

4.4.3 功能模块优化设计原则

■ 耦合分类：

■ (1) 数据耦合

- 如果两个模块之间存在调用与被调用关系；
- 并且还存在着**数据通讯**，即通过参数交换信息，而且交换的信息仅仅是数据，则称为数据耦合。
- 示例：

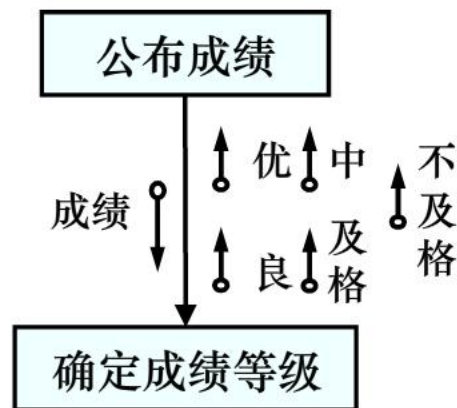
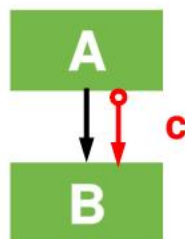


4.4.3 功能模块优化设计原则

■ 耦合分类：

■ (2)控制耦合

- 如果两个模块之间存在调用与被调用关系；
- 而且模块A向模块B传递的信息控制了模块B的内部逻辑，那么模块A、B之间的耦合称为控制耦合。
- 示例：

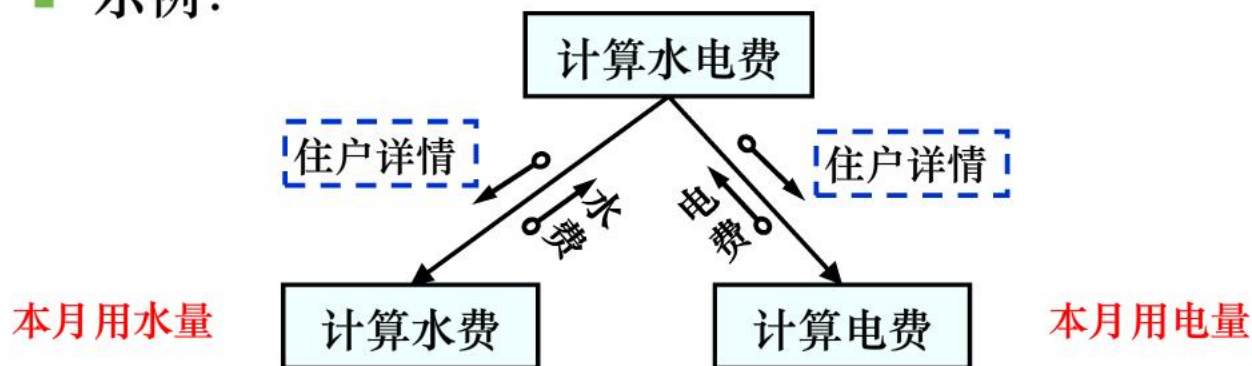
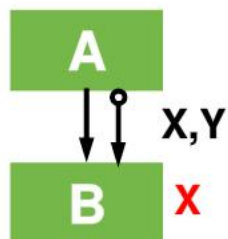


4.4.3 功能模块优化设计原则

■ 耦合分类：

■ (3)特征耦合

- 如果两个模块之间存在调用与被调用关系；
- 同时还存在着数据通讯，并且B模块仅使用了A模块传递的数据结构中的一部分数据元素，则模块A、B之间的耦合称为特征耦合。
- 示例：

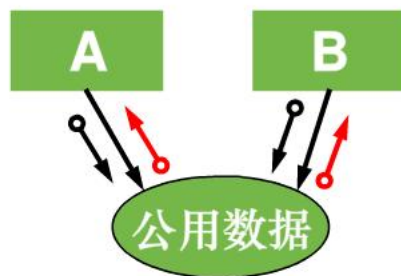


4.4.3 功能模块优化设计原则

■ 耦合分类：

■ (4)公共环境耦合

- 如果两个模块或多个模块通过一个公共数据环境相互作用时，它们之间的耦合称为公共环境耦合。
- ①数据耦合的形式：
 - 一个模块往公共环境送数据，
 - 另一个模块从公共环境取数据。
- ②介于数据耦合和控制耦合之间：
 - 两个模块都既往公共环境送数据又从里面取数据。这种耦合比较紧密。

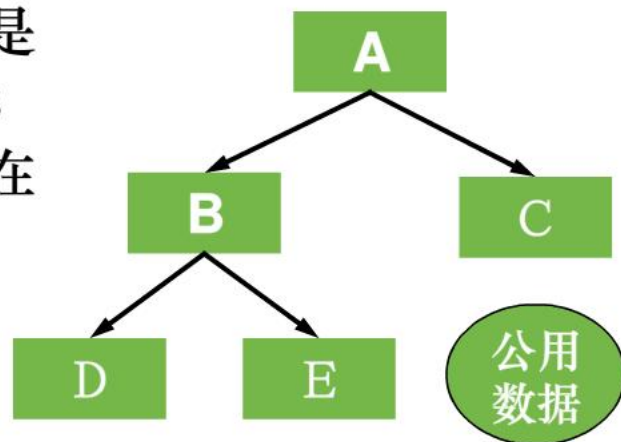


4.4.3 功能模块优化设计原则

- 耦合分类：

- (4)公共环境耦合

- 示例：
- 假设有一个全局变量X是这5个模块的公用数据；
- 那么模块E和C之间存在着公共耦合。



全局变量：X

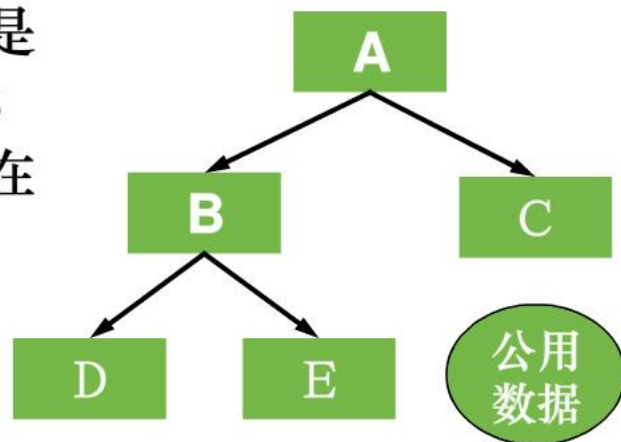
4.4.3 功能模块优化设计原则

- 耦合分类：

- (5)公共环境耦合

- 示例：

- 假设有一个全局变量X是这5个模块的公用数据；
- 那么模块E和C之间存在着公共耦合。



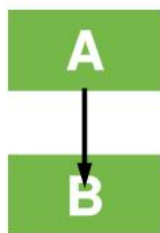
全局变量：X

4.4.3 功能模块优化设计原则

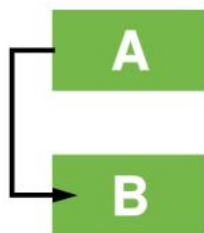
■ 耦合分类：

■ (5)内容耦合 有下述几种情况：

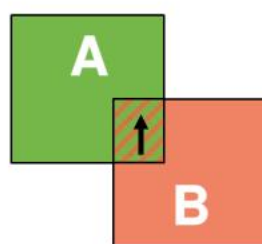
- ①一个模块访问另一个模块的内部数据；
- ②一个模块不通过正常入口而转移到另一个模块的内部；
- ③两个模块有一部分程序代码重叠(只出现在汇编程序中)；
- ④一个模块有多个入口（一个模块有多种功能）。



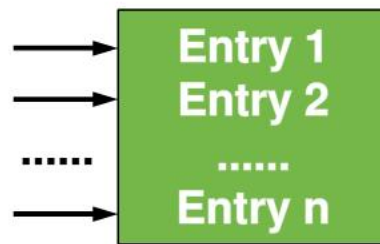
①



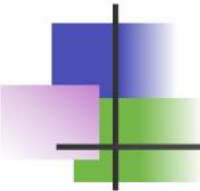
②



③

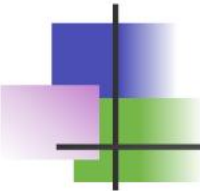


④



4.4.3 功能模块优化设计原则

- 耦合小结：
 - **耦合**是影响软件复杂程度的一个重要因素。
 - 应该采取下述设计原则：
 - 尽量使用数据耦合，
 - 少用控制耦合和特征耦合，
 - 限制公共环境耦合的范围，
 - 完全不用内容耦合。
- 思考题：
 - 请同学们例举出各种耦合的实例。



4.4.3 功能模块优化设计原则

■ 2. 模块的内聚性原则

■ 内聚：

- 标志一个模块内各个元素彼此结合的紧密程度。
- 即一个模块内部的各个部分其处理动作的组合强度。

■ 理想内聚的模块：只做一件事情。

X、Y、Z

■ 内聚与耦合的关系：

- 高内聚意味着模块间的松耦合。
- 高内聚>松耦合

高内聚→松耦合

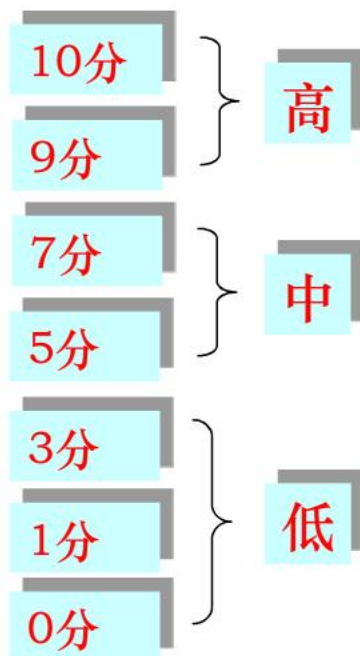
求和

4.4.3 功能模块优化设计原则

■ 2. 模块的内聚性原则

■ 内聚分类：

- 功能内聚
- 顺序内聚
- 通信内聚
- 过程内聚
- 时间内聚
- 逻辑内聚
- 偶然内聚



4.4.3 功能模块优化设计原则

■ 内聚分类

■ (1) 功能内聚

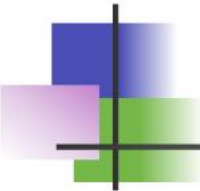
10分

- 如果模块内所有处理元素属于一个整体，完成一个单一的功能，则称为功能内聚。
- 功能内聚是最高程度的内聚。

■ 示例

- 计算水费
- 执行修改存盘
- 计算机自动排课
- 计算某工厂职工的实发工资





4.4.3 功能模块优化设计原则

■ 内聚分类

■ (2)顺序内聚

9分

- 如果一个模块内的处理元素和同一个**功能密切相关**，而且这些处理必须顺序执行(通常一个处理元素的输出数据作为下一个处理元素的输入数据)，则称为顺序内聚。

■ 示例

- 查询并显示查询结果
 - 先获得查询条件，再执行显示功能
- 登录模块

4.4.3 功能模块优化设计原则

■ 内聚分类

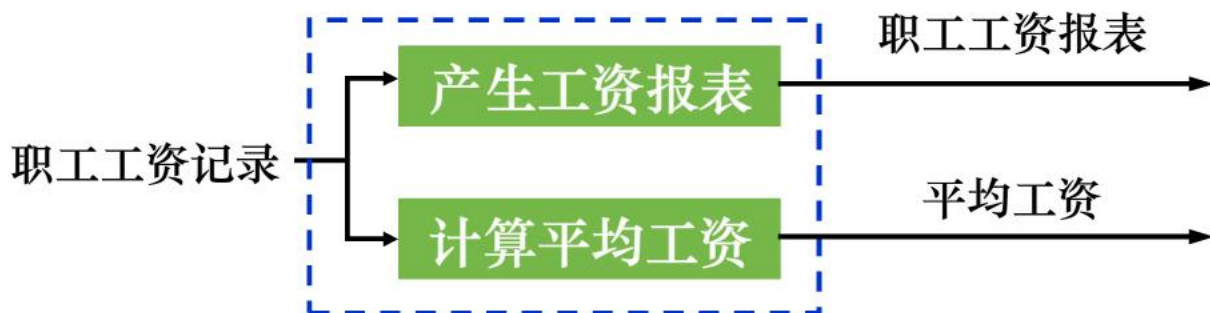
■ (3)通信内聚

7分

- 如果模块中所有元素都使用同一个输入数据和(或)产生同一个输出数据，则称为通信内聚。

■ 示例

- 一个模块，是某工厂“产生职工工资报表和计算职工的平均工资”，他们都使用职工的工资记录信息。



4.4.3 功能模块优化设计原则

■ 内聚分类

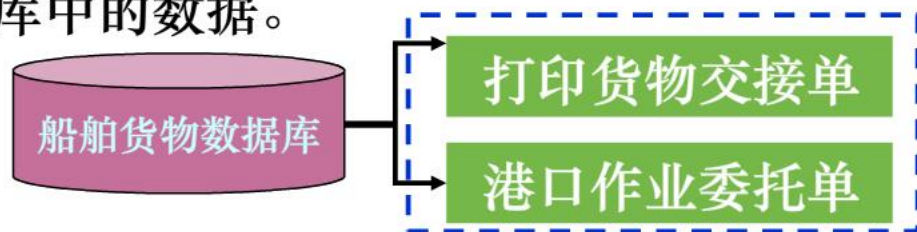
■ (3)通信内聚

7分

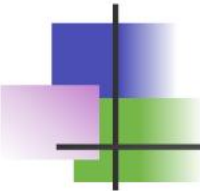
- 如果模块中所有元素都使用同一个输入数据和(或)产生同一个输出数据，则称为通信内聚。

■ 示例

- 一个模块有两个动作，他们是“打印货物交接单”和“港口作业委托单”。这两个动作都使用了船舶货物数据库中的数据。



- 这两个例子都是使用了同一个数据，而产生了不同的输出。



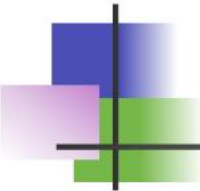
4.4.3 功能模块优化设计原则

■ 内聚分类

■ (4)过程内聚

5分

- 如果一个模块内部各组成部分的处理动作各不相同，彼此也没有什么关系，但它们都受同一个控制流支配，以决定它们的执行顺序，则称为过程内聚。
- 如果一个模块内的处理元素是**相关**的，而且必须以特定次序执行，则称为过程内聚。
- 使用程序流程图设计软件时，常常通过研究流程图确定模块的划分，这样得到的往往是过程内聚的模块。



4.4.3 功能模块优化设计原则

■ 内聚分类

■ (4)过程内聚

5分

■ 示例

- 如果将“学籍处理”设计成一个模块，则该模块接受的是同一个控制流——学籍处理申请的支配，其内部的各个处理动作执行着各不相同的运动作：基本信息管理、奖励信息管理、处分信息管理、变动信息管理、课外活动管理。

■ 提高内聚的方法：

- 将模块**分解**为**功能**模块。

4.4.3 功能模块优化设计原则

■ 内聚分类

■ (4)过程内聚

5分

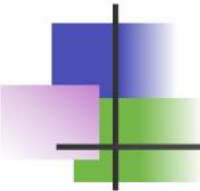
■ 示例

- 如果一个模块中的读入成绩单与查询成绩单必须顺序执行。
- 如果一个模块中的统计成绩与打印成绩必须顺序执行。

■ 提高内聚的方法：

- 将模块**分解**为**功能**模块。





4.4.3 功能模块优化设计原则

■ 内聚分类

■ (5)时间内聚

3分

- 如果一个模块包含的任务必须在同一段时间内执行，就叫时间内聚。

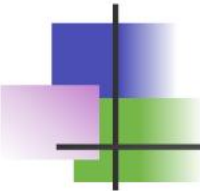
■ 示例

- “初始化”模块。
- 紧急故障处理系统。
- 学生选课

- 不同专业（不同的任务）的学生选同一节课。

■ “初始化”模块。

- 在系统或程序启动时必须做完一系列的初始化工作，如变量初始化、数组初始化等，它与时间有关，但执行次序不重要，所处理的变量、数组与其后调用的许多程序有关。



4.4.3 功能模块优化设计原则

■ 内聚分类

■ (6)逻辑内聚

1分

- 如果一个模块完成的任务在逻辑上属于相同或相似的一类，则称为逻辑内聚。

■ “逻辑相似”

- 假设学生回家，可以乘火车、汽车、飞机，这几种方式在逻辑上相似，都是交通工具，但每个交通工具又完全不同，而且只能选择其中一种，即所执行的功能不同。

4.4.3 功能模块优化设计原则

■ 内聚分类

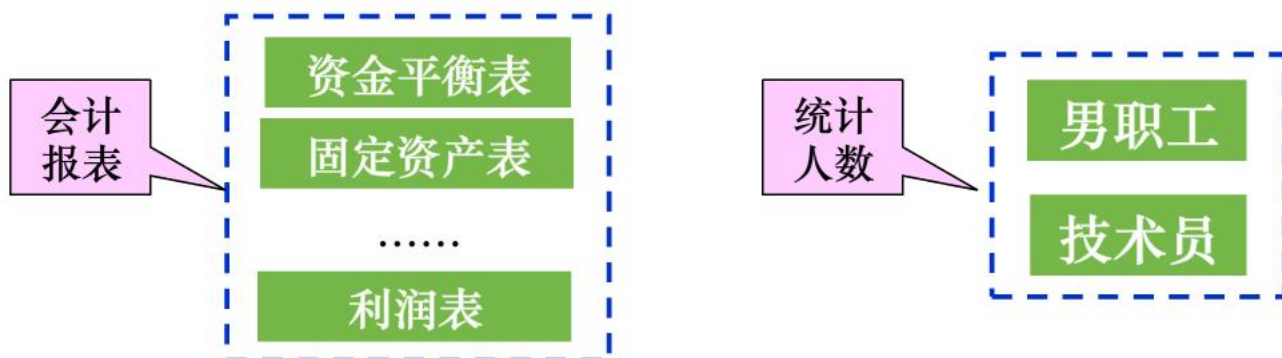
■ (6)逻辑内聚

1分

- 如果一个模块完成的任务在逻辑上属于相同或相似的一类，则称为逻辑内聚。

■ 示例

- “审核会计报表”模块
- 一个工厂需要统计男职工人数和技术员人数。



4.4.3 功能模块优化设计原则

■ 内聚分类

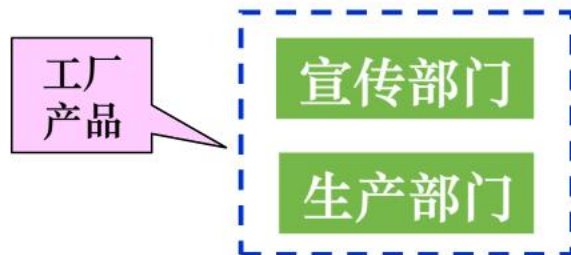
■ (7)偶然内聚

0分

- 如果一个模块内的各个任务间的关系很松散，则称为偶然内聚。
- 如果一个模块内的各部分的处理动作彼此没有任何关系，则称这些部分的组合为偶然内聚。

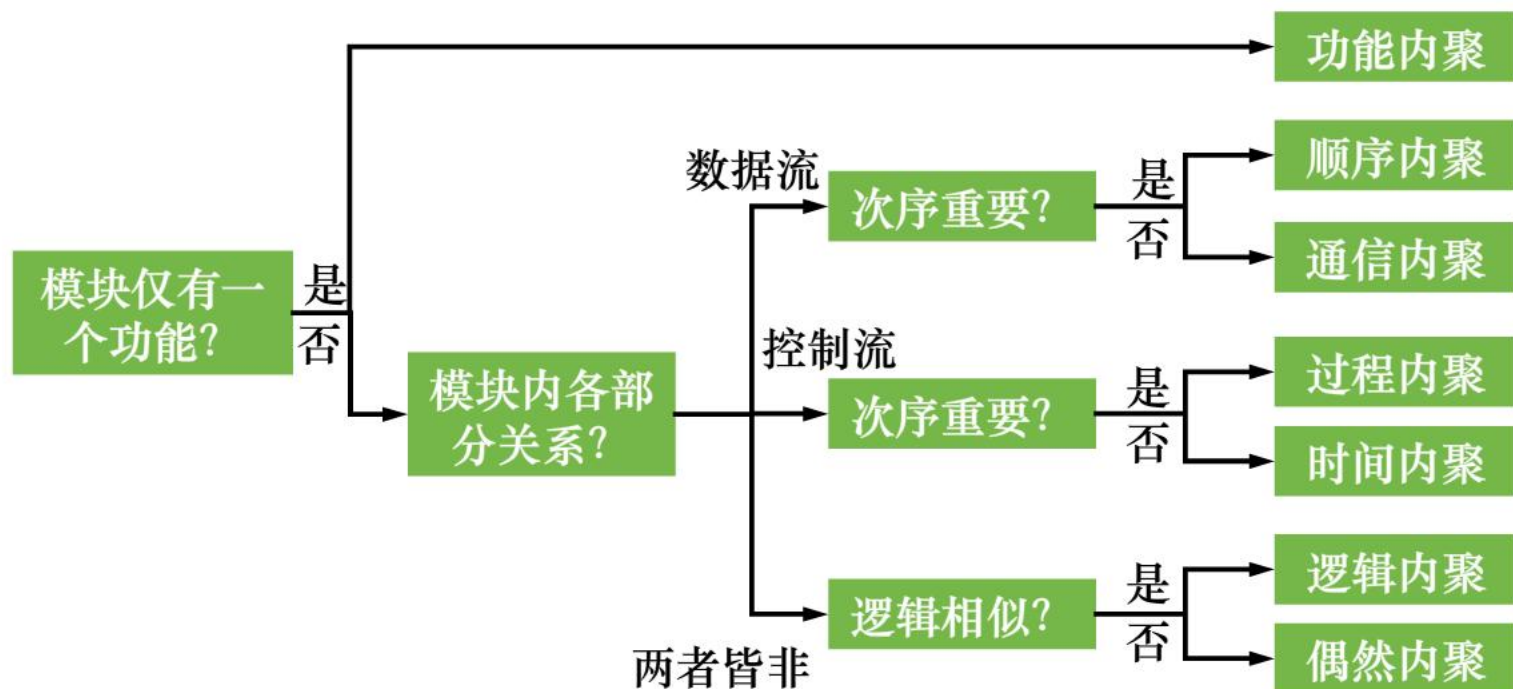
■ 示例

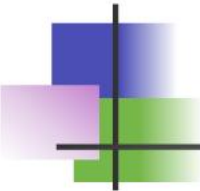
- 一个工厂（产品）的宣传部门与生产部门。



4.4.3 功能模块优化设计原则

■ 内聚类型判断树





4.4.3 功能模块优化设计原则

■ 3.模块的分解原则

■ 模块的分解：

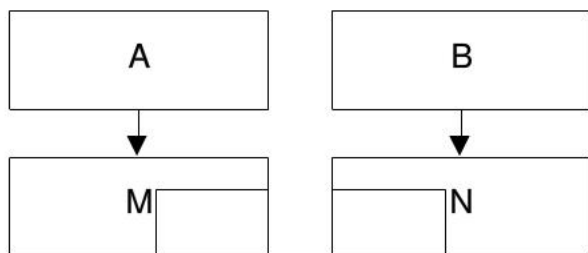
- 指把一个模块分解成若干个从属于它的新模块。

■ 分解原则：

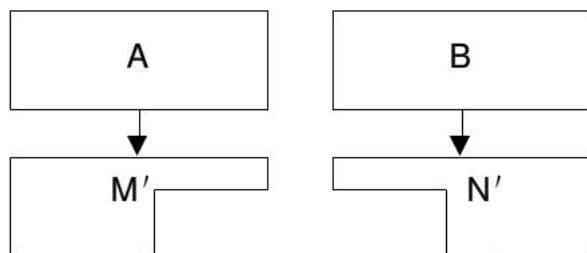
- 主要是按功能进行分解，直到无法做出明确的功能定义为止。
- 在分解时既要考虑到模块的内聚性，又要考虑到模块之间的耦合程度，并在这两者之间选择一个最佳的方案。

4.4.3 功能模块优化设计原则

- 3.模块的分解原则
- 示例:

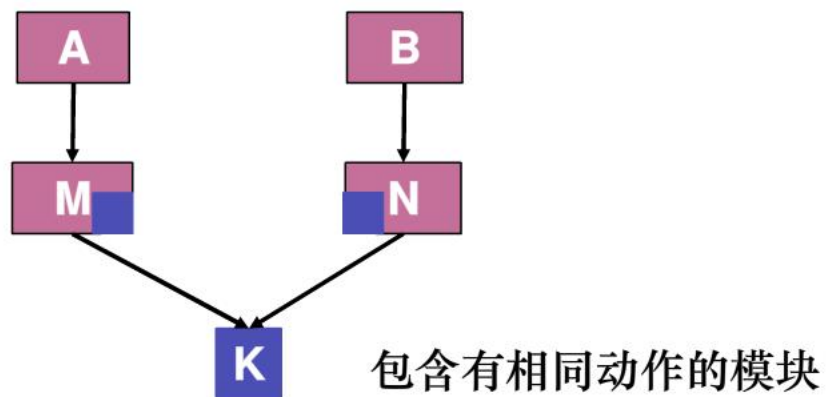


(a)



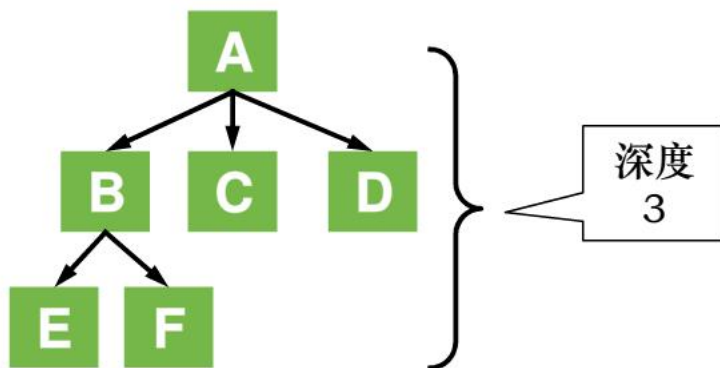
(b)

模块分解示意图



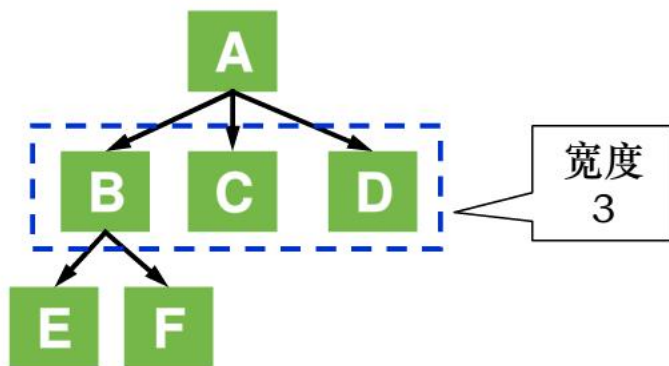
4.4.3 功能模块优化设计原则

- 4.模块的深度、宽度、扇出、扇入原则
- 深度
 - 软件结构中控制的层数。
 - 层数过多，有可能许多管理模块过分简单，应考虑合并。



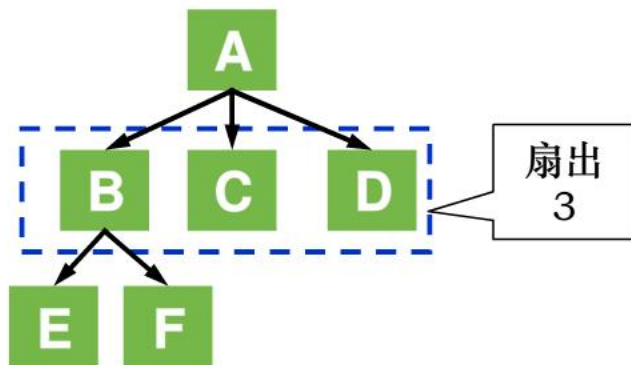
4.4.3 功能模块优化设计原则

- 4.模块的深度、宽度、扇出、扇入原则
- 深度
- 宽度
 - 软件结构中同一层次上模块总数的最大值。
 - 影响因素：模块的扇出。



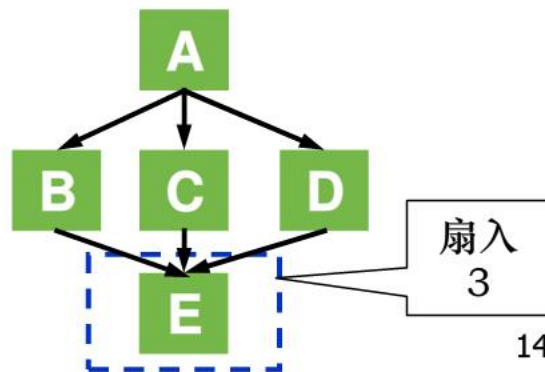
4.4.3 功能模块优化设计原则

- 4.模块的深度、宽度、扇出、扇入原则
- 扇出 (Fan_Out)
 - 一个模块直接控制（调用）的模块数目。
 - 或一个模块对它的直属下级模块的控制范围。
 - 扇出系数：指其直属下级模块的个数。
 - 平均扇出：3—4，上限为5-9



4.4.3 功能模块优化设计原则

- 4.模块的深度、宽度、扇出、扇入原则
- 扇出 (Fan_Out)
- 扇入 (Fan_In)
 - 一个模块有多少个上级模块直接调用它。
 - 扇入系数：指其直接上级模块的个数。
 - 扇入越大，由共享该模块的上级模块数目越多。



4.4.3 功能模块优化设计原则

- 4.模块的深度、宽度、扇出、扇入原则
- 扇出 (Fan_Out)
- 扇入 (Fan_In)
- 小结

- 很好的软件结构
 - 顶层扇出比较高;
 - 中层扇出较少;
 - 底层高扇入 (底扇入到公共的实用模块中去)。

